

Who can do what in REEP

Every role's features, and for each one which of create, read, update and delete that role is offered, with the API endpoints, screens, code, tables and AWS services behind it. Every feature is shown open.

379 endpoints 149 role-features 112 tables taken 2026-10-03 at f67b27b

STUDENT	37 features
FACULTY (role MENTOR)	42 features
ALUMNI	11 features
MAIN ADMIN (role ADMIN, the placement office)	62 features
EVERY SIGNED-IN ACCOUNT	5 features
PUBLIC - no session	9 features
Stack & AWS	18 table groups

STUDENT

A student signed in: their own records only.

GATE `_require_student` / `require_student` — users.role STUDENT, own rows only;
most screens also pass a per-student feature switch.

37 features **95** endpoints features that **15** create **35** read **10** update **3** delete
3 action

Student records & roster 4 features

Academic records

€

R

U

Ð

3 endpoints · /student/records

R

GET

/api/student/academics

view my academic history

R

GET

/api/student/attendance

view my attendance

R

GET

/api/student/results

view my semester results

SCREENS

/student/records

/student/resume

CODE

routers/student.py

GATE

get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES

academic_gaps

academic_qualifications

attendance_records

semester_results

subject_marks

AWS

none

My uploads

C R  D

4 endpoints · /student/uploads

- C** **POST** /api/student/uploads
upload a document
- R** **GET** /api/student/uploads
list my uploads
- R** **GET** /api/student/uploads/{upload_id}/file
download my file
- D** **DELETE** /api/student/uploads/{upload_id}
delete my upload

SCREENS /student/uploads /student/skilling /student/resume

CODE routers/student.py

GATE get_current_session + _require_student (role STUDENT and a studentId claim) ·
get_current_session + _require_student (role STUDENT and a studentId claim) +
require_feature(student.uploads)

TABLES archived_documents feature_overrides uploads

AWS EFS /data volume · S3 documents archive

Placement profile

 R U 

2 endpoints · /student/profile

- R** **GET** /api/student/profile
view my profile
- U** **PUT** /api/student/profile
edit my profile

SCREENS /student/profile /student/resume

CODE routers/student.py

GATE get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES academic_courses academic_specializations cohorts colleges departments mentors
student_profiles students users

AWS none

Student home

€ R U D

2 endpoints · /student/time-log

- R GET /api/student/dashboard
view my dashboard summary
- R GET /api/student/overview
load my home page

Programme (ledger, English, milestones) 9 features

SCREENS /student/time-log /student/profile /student/resume /student /student/mentor-log

English Baseline

C R U D

3 endpoints · /student/english

- C POST /api/student/english-baseline/start
start English assessment
- R GET /api/student/english-baseline
view English baseline
- R GET /api/student/english-baseline/report
download English report PDF

SCREENS /student/english

CODE routers/student_programme.py

GATE _require_student · _require_student + require_feature(student.english)

TABLES english_baseline_sections english_baselines feature_overrides students users

AWS PDF render (in process)

Focus sessions (lab check-in)

C R U D

3 endpoints · API only

- C POST /api/student/checkin
start focus session API ONLY
- R GET /api/student/focus
list my focus sessions API ONLY
- U POST /api/student/checkout/{session_id}
end focus session API ONLY

SCREENS none for this role (API only)

CODE routers/student.py

GATE get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES courses lab_sessions

AWS none

Login streak

€ R U D

1 endpoint · API only

R GET /api/student/streak
view my login streak API ONLY

SCREENS none for this role (API only)

CODE routers/student.py

GATE get_current_session only (no _require_student; keyed on session userId)

TABLES login_days

AWS none

My courses

€ R U D

1 endpoint · /student/courses

R GET /api/student/courses
list my courses

SCREENS /student/courses

CODE routers/student.py

GATE get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES courses enrollments

AWS none

My schedule

€ R U D

1 endpoint · API only

R GET /api/student/schedule
list my upcoming schedule API ONLY

SCREENS none for this role (API only)

CODE routers/student.py

GATE get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES schedule_items

AWS none

Next actions



1 endpoint · API only

R **GET** `/api/student/next-actions`

list my next actions **API ONLY**

SCREENS none for this role (API only)

CODE `routers/student.py`

GATE `get_current_session + _require_student (role STUDENT and a studentId claim)`

TABLES `courses` `enrollments` `interview_evaluations` `interview_sessions` `resume_profiles`
`skill_claims` `skills` `student_profiles` `student_skills` `swoc_entries`

AWS none

Programme stage cards



1 endpoint · /student

R **GET** `/api/student/programme`

view programme stages

SCREENS `/student`

CODE `routers/student_programme.py`

GATE `_require_student`

TABLES `english_baseline_sections` `english_baselines` `student_milestones`

AWS none

Time Allocation Ledger

C R U D

6 endpoints · /student/time-log

- R** **GET** /api/student/ledger
view ledger day
- R** **GET** /api/student/ledger/history
list ledger history
- R** **GET** /api/student/timesheet
view weekly skilling hours
- U/C** **PUT** /api/student/ledger
start or save ledger day
- U** **POST** /api/student/ledger/copy-yesterday
copy yesterday's ledger
- U** **POST** /api/student/ledger/submit
submit ledger day

SCREENS /student/time-log

CODE routers/student.py routers/student_programme.py

GATE _require_student + require_feature(student.time_log) · get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES feature_overrides students time_ledger_cells time_ledger_days time_sheet_entries

AWS none

Time sheet (legacy, no screen)

C R U D

1 endpoint · API only

- C** **POST** /api/student/timesheet
log activity minutes **API ONLY**

SCREENS none for this role (API only)

CODE routers/student.py

GATE get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES time_sheet_entries

AWS none

Leaderboards

€

R

U

🔗

2 endpoints · /student/leaderboards

R

GET

/api/student/leaderboards

view batch leaderboard

U

PUT

/api/student/leaderboard-visibility

hide or show me

SCREENS

/student/leaderboards

CODE

routers/student.py

GATE

get_current_session + _require_student (role STUDENT and a studentId claim) ·
get_current_session + _require_student (role STUDENT and a studentId claim) +
require_feature(student.leaderboards)

TABLES

academic_courses academic_specializations certification_progress cohorts departments
feature_overrides interview_score_summaries login_days mock_attempts semester_results
student_badges student_profiles students users

AWS

none

My skills

€

R

U

🔗

2 endpoints · /student/resume

R

GET

/api/student/skills

list my skills

R

GET

/api/student/skills/catalogue

browse skill catalogue API ONLY

SCREENS

/student/resume

CODE

routers/student.py

GATE

get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES

skills student_skills

AWS

none

Skill claims (legacy)

C R U D

2 endpoints · /student/resume

C **POST** /api/student/skill-claims
file a skill claim **API ONLY**

R **GET** /api/student/skill-claims
list my skill claims

SCREENS /student/resume

CODE routers/student.py

GATE get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES skill_claims skills uploads

AWS none

Skill recommendations

€ R U D

1 endpoint · API only

R **GET** /api/student/recommendations
list recommended skills **API ONLY**

SCREENS none for this role (API only)

CODE routers/student.py

GATE get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES resume_profiles skills student_skills

AWS none

Skills & badges

C R U D

5 endpoints · /student/skilling

- C** **POST** /api/student/badges/{code}/evidence
submit badge claim
- C** **POST** /api/student/badges/{code}/start
start a badge **API ONLY**
- R** **GET** /api/student/badges
view my badge dashboard
- R** **GET** /api/student/badges/leaderboards
view badge leaderboards **API ONLY**
- R** **GET** /api/student/growth
view my capability growth **API ONLY**

SCREENS /student/skilling

CODE routers/badges.py

GATE _require_student + require_feature(student.leaderboards) · _require_student + require_feature(student.skilling)

Interviews

Interview policy

€ R U D

1 endpoint · /student/assistant

- R** **GET** /api/interview/policy
view my interview policy

SCREENS /student/assistant (agent dock on every screen)

CODE routers/interview_policy.py

GATE session role = STUDENT and studentId present

TABLES academic_courses academic_specializations cohorts departments interview_cap_resets
interview_consentss interview_policies interview_sessions interview_tracks students

AWS none

Mock interview

€ R U D A

2 endpoints · /student/assistant

R **GET** /api/interview/status
check interview availability

A **WS** /api/interview
run live mock interview

SCREENS /student/assistant (agent dock on every screen)

CODE routers/interview.py

GATE get_current_session; available only to role STUDENT or ADMIN (_is_rehearsal), any other role gets 200 available=false · get_ws_session cookie + Origin check; role STUDENT or ADMIN (_is_rehearsal) else close 1008; student.assistant feature switch (close 4016); live consent (4013); concurrency and daily/attempt caps (1013/4012/4015)

TABLES archived_documents cohorts conversations feature_overrides interview_bank_questions interview_cap_resets interview_consent interview_evaluations interview_policies interview_score_summaries interview_sessions interview_tracks interview_turns messages students

AWS Bedrock Nova 2 Sonic · EFS /data volume · Sentry

Mock interview consent

C R U D

2 endpoints · /student/assistant

C **POST** /api/interview/consent
acknowledge interview terms

R **GET** /api/interview/consent
read my interview consent

SCREENS /student/assistant

CODE routers/interview_records.py

GATE get_current_session (no role check) · get_current_session + role == STUDENT (403 otherwise); version must match INTERVIEW_CONSENT_VERSION

TABLES interview_consent interview_policies students

AWS none

My mock interviews



7 endpoints · /student/interviews

- R** **GET** /api/interview/progress
view my interview trend **API ONLY**
- R** **GET** /api/interview/sessions
list my interviews
- R** **GET** /api/interview/sessions/{session_id}
open my interview
- R** **GET** /api/interview/sessions/{session_id}/report
view my scorecard
- R** **GET** /api/interview/sessions/{session_id}/transcript
view my transcript
- R** **GET** /api/interview/sessions/{session_id}/views
see who viewed my interview **API ONLY**
- R** **GET** /api/student/mocks
list my mock attempts **API ONLY**

SCREENS /student/interviews

CODE routers/interview_records.py routers/student.py

GATE _own_student_id (403 unless role STUDENT; subject from session cookie) · _own_student_id (403 unless role STUDENT; subject from session cookie) + _session_of_student_or_404 · get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES interview_evaluations interview_score_summaries interview_sessions interview_turns mock_attempts redesign_audit_events users

AWS none

Mentoring & SWOC 3 features

Faculty / TPO Log (meeting notes)

C R U D

2 endpoints · /student/mentor-log

C **POST** /api/student/mentor-meetings/request

request mentor meeting

R **GET** /api/student/mentor-meetings

list my mentor meetings

SCREENS /student/mentor-log /student/resume

CODE routers/student_programme.py

GATE _require_student + require_feature(student.mentor_log)

TABLES feature_overrides mentor_notes mentors redesign_mentor_notebook_entries schedule_items students users

AWS none

Published notebook entries

€ R U D

1 endpoint · API only

R **GET** /api/v1/student/mentor-notebook

read published notes API ONLY

SCREENS none for this role (API only)

CODE routers/redesign.py

GATE student_identity (role STUDENT + session studentId)

TABLES redesign_mentor_notebook_entries

AWS none

SWOC board

€ R U D

2 endpoints · /student/mentor-log

R **GET** /api/student/swoc

view my SWOC board API ONLY

U **POST** /api/student/swoc/{entry_id}/acknowledge

acknowledge a SWOC line

SCREENS /student/mentor-log

CODE routers/student.py

GATE get_current_session + _require_student (role STUDENT and a studentId claim)

TABLES swoc_entries users

AWS none

Leave 5 features

College leave calendar



1 endpoint · API only

R **GET** `/api/leaves/calendar`
view my college calendar **API ONLY**

SCREENS none for this role (API only)

CODE `routers/leave_policy.py`

GATE `get_current_session (caller's own college only)`

TABLES `academic_calendar` `cohorts` `departments` `students` `users`

AWS none

Leave alternate arrangements



2 endpoints · API only

R **GET** `/api/leaves/{leave_id}/alternate`
view alternate table **API ONLY**

U **POST** `/api/leaves/{leave_id}/alternate/assign`
name covering colleague **API ONLY**

SCREENS none for this role (API only)

CODE `routers/leave_alternate.py`

GATE `get_current_session; applicant OR leave._assert_can_decide (require_mentor, then ADMIN or has_capability(admin.leave_approvals)) · get_current_session; applicant only (requester_user_id = caller, else flattened 404); request must be SUBMITTED/FIRST_APPROVED`

TABLES `capability_grants` `leave_requests` `users`

AWS none

Leave attachments

C R U D

4 endpoints · API only

- C** **POST** `/api/leaves/{leave_id}/attachments`
attach document to leave **API ONLY**
- R** **GET** `/api/leaves/{leave_id}/attachments`
list leave attachments **API ONLY**
- R** **GET** `/api/leaves/{leave_id}/attachments/{attachment_id}/file`
download leave attachment **API ONLY**
- D** **DELETE** `/api/leaves/{leave_id}/attachments/{attachment_id}`
remove leave attachment **API ONLY**

SCREENS none for this role (API only)

CODE `routers/leave_attachments.py`

GATE `get_current_session; applicant (requester_user_id == caller) OR leave._assert_can_decide (require_mentor + ADMIN, or has_capability(admin.leave_approvals)); every refusal flattened to 404 · get_current_session; applicant (requester_user_id == caller) OR leave._assert_can_decide (require_mentor + ADMIN, or has_capability(admin.leave_approvals)); every refusal flattened to 404; then _may_delete (the uploader, or role ADMIN for any file)`

TABLES `archived_documents` `capability_grants` `leave_attachments` `leave_requests` `users`

AWS EFS /data volume · S3 documents archive

Leave policy & allowances

C R U D

2 endpoints · API only

- R** **GET** `/api/leaves/balances`
view my leave allowances **API ONLY**
- R** **GET** `/api/leaves/{leave_id}/balance`
view applicant's allowances **API ONLY**

SCREENS none for this role (API only)

CODE `routers/leave_policy.py`

GATE `get_current_session (own user id only) · requester themselves, else _assert_can_decide (require_mentor; ADMIN, or has_capability(admin.leave_approvals); other refusals flattened to 404)`

TABLES `capability_grants` `leave_balances` `leave_requests`

AWS none

My own leave (apply / withdraw)

C R U D

4 endpoints · API only

- C** **POST** `/api/leaves`
apply for leave **API ONLY**
- R** **GET** `/api/leaves/mine`
list my leave **API ONLY**
- R** **GET** `/api/leaves/{leave_id}/paper.pdf`
download leave PDF **API ONLY**
- U** **POST** `/api/leaves/{leave_id}/cancel`
withdraw my leave **API ONLY**

SCREENS none for this role (API only)

CODE `routers/leave.py` `routers/leave_paper.py`

GATE `get_current_session (+ leave_policy.submit_refusal balance/overlap checks) ·`
`get_current_session (requester_user_id = caller) · get_current_session (requester_user_id =`
`caller, else 404) · requester themselves, else _assert_can_decide (require_mentor; ADMIN, or`
`has_capability(admin.leave_approvals); other refusals flattened to 404)`

TABLES `academic_calendar` `capability_grants` `leave_balances` `leave_requests` `mail_logs`
`staff_signatures` `students` `users`

AWS EFS /data volume · PDF render (in process) · Amazon SES mail

Jobs & placement 4 features

Jobs board

C R U D

2 endpoints · `/student/jobs`

- C** **POST** `/api/student/jobs/{job_id}/apply`
apply to a job
- R** **GET** `/api/student/jobs`
browse eligible jobs

SCREENS `/student/jobs` `/student/resume`

CODE `routers/student.py`

GATE `get_current_session + _require_student (role STUDENT and a studentId claim) +`
`require_feature(student.jobs)`

TABLES `academic_gaps` `feature_overrides` `job_applications` `jobs` `placement_criteria`
`semester_results` `skills` `student_skills` `students`

AWS none

My offers

C R U D

3 endpoints · API only

- C** **POST** `/api/student/offers`
record an offer **API ONLY**
- R** **GET** `/api/student/offers`
list my offers **API ONLY**
- U** **POST** `/api/student/offers/{offer_id}/submit`
submit offer for approval **API ONLY**

SCREENS none for this role (API only)

CODE `routers/student.py`

GATE `get_current_session + _require_student (role STUDENT and a studentId claim)`

TABLES `jobs` `placement_offers`

AWS none

Placement readiness

C R U D

1 endpoint · API only

- R** **GET** `/api/student/placement-readiness`
view my readiness score **API ONLY**

SCREENS none for this role (API only)

CODE `routers/student.py`

GATE `get_current_session + _require_student (role STUDENT and a studentId claim)`

TABLES `attendance_records` `certification_progress` `cohorts` `interview_score_summaries`
`placement_criteria` `resume_profiles` `semester_results` `student_profiles` `students`

AWS none

Resume builder

C R U D

5 endpoints · /student/resume

C	POST	/api/student/resume/generate	generate a resume
R	GET	/api/student/resume	list my resume versions
R	GET	/api/student/resume-profile	load resume builder state
R	GET	/api/student/resume/{resume_id}/pdf	download resume PDF
U	PUT	/api/student/resume-profile	save resume builder

SCREENS /student/resume

CODE routers/student.py

GATE get_current_session + _require_student (role STUDENT and a studentId claim) ·
get_current_session + _require_student (role STUDENT and a studentId claim) +
require_feature(student.resume) + llm_rate_limited

TABLES academic_qualifications feature_overrides resume_profiles resumes semester_results
skill_claims skills student_profiles student_skills uploads

AWS EFS /data volume · LLM (Bedrock / OpenAI-compatible) · PDF render (in process)

REEP Agent

8 endpoints · /student/agent

C

POST

/api/agent/feedback

rate an answer

R

GET

/api/agent/history

read my chat history

R

GET

/api/agent/knowledge/search

search knowledge base

API ONLY

R

GET

/api/agent/runs

list my agent runs

API ONLY

D

DELETE

/api/agent/conversation

clear my conversation

A

POST

/api/agent/ask

ask the agent

A

POST

/api/agent/chat

chat with assistant

API ONLY

A

POST

/api/agent/chat/stream

stream assistant reply

API ONLY

SCREENS

/student/agent

(agent dock on every screen)

/student/assistant

CODE

routers/agent.py

GATE

_require_agent_access: require_feature(student.agent) for STUDENT; llm_rate_limited · get_current_session (caller's own runs only) · get_current_session (caller's own thread only) · get_current_session + caller must own the agent run (404 otherwise) · get_current_session + inline role = STUDENT

TABLES

agent_runs

assistant_feedback

conversations

feature_overrides

knowledge_chunks

knowledge_documents

messages

student_profiles

students

AWS

Embeddings (pgvector KB) · LLM (Bedrock / OpenAI-compatible)

REEP roles & CRUD map · 19 / 103

Voice platform call sessions

€ R U Ð

3 endpoints · API only

- R

GET

/api/platform/calls

list call sessions

API ONLY
- R

GET

/api/platform/calls/{session_id}

view call detail

API ONLY
- U

POST

/api/platform/calls/{session_id}/close

close call, upload recording

API ONLY

SCREENS none for this role (API only)

CODE voice_platform/api/calls.py

GATE `get_current_session (any signed-in session); role = ADMIN lists every call, every other role is filtered to user_id = session.userId · get_current_session + _can_read (owner | role ADMIN | MENTOR via _assert_can_access_student on the linked interview's student); presigned recording URL only if has_capability(admin.interview_audio) · get_current_session + owner (row.user_id = session.userId) or role ADMIN, else 403; 409 if running on another worker`

TABLES capability_grants interview_sessions interview_turns mentors platform_call_sessions platform_recording_policies students

AWS CloudWatch metrics · DynamoDB voice sessions · EFS /data volume · S3 call recordings · Sentry

Voice platform interview socket

€ R U Ð A

2 endpoints · API only

- A

WS

/api/platform/media-bridge

run platform mock interview

API ONLY
- A

WS

/ws/media-bridge

run platform mock interview

API ONLY

SCREENS none for this role (API only)

CODE voice_platform/api/media_bridge.py

GATE `Origin = settings.web_origin + get_ws_session + role = STUDENT with studentId (else _CLOSE_NOT_A_STUDENT) + _LIMITER + _open_records (consent 4013, caps 4012/4015)`

TABLES archived_documents conversations interview_cap_resets interview_consents interview_evaluations interview_policies interview_score_summaries interview_sessions interview_turns messages platform_call_sessions platform_candidates platform_questions platform_recording_policies platform_specializations platform_time_limits

AWS Bedrock Nova 2 Sonic · CloudWatch metrics · DynamoDB voice sessions · EFS /data volume · S3 call recordings · Sentry

FACULTY (role MENTOR)

A faculty member (role MENTOR): their own things, plus their mentee group while they mentor somebody. Console screens only when the Main Admin grants them.

GATE `require_mentor` (admits MENTOR and ADMIN) + rule 2
`_assert_can_access_student` : own mentee group only. Group functions (mentees, notebook, verifications) are DERIVED while they mentor somebody; console screens only by a Governance grant.

25 features 75 endpoints features that 9 create 23 read 10 update 6 delete
1 action

Programme (ledger, English, milestones) 3 features

English Baseline (staff view)

1 endpoint · API only

R

GET

/api/mentor/students/{student_id}/english-baseline

view mentee English baseline

API ONLY

SCREENSnone for this role (API only)

CODErouters/mentee_records.py

GATERequire_capability(mentor.mentees) + _assert_can_access_student(allow_handover=True) · capability mentor.mentees

TABLEScapability_grants english_baseline_sections english_baselines mentors students

AWSnone

Login streak

1 endpoint · API only

R

GET

/api/student/streak

view my login streak

API ONLY

SCREENSnone for this role (API only)

CODErouters/student.py

GATEget_current_session only (no _require_student; keyed on session userId)

TABLESlogin_days

AWSnone

Time Allocation Ledger (staff view)

€ R U D

2 endpoints · API only

R GET /api/mentor/students/{student_id}/ledger

view mentee ledger day API ONLY

R GET /api/mentor/students/{student_id}/ledger/summary

view mentee ledger summary API ONLY

SCREENS none for this role (API only)

CODE routers/mentee_records.py

GATE require_capability(mentor.mentees) + _assert_can_access_student(allow_handover=True) ·
capability mentor.mentees

TABLES capability_grants mentors students time_ledger_cells time_ledger_days

AWS none

Skills, badges & certifications 6 features

Badge awards

C R U D

1 endpoint · API only

C POST /api/mentor/students/{student_id}/badges/{code}/award

manually award badge API ONLY

SCREENS none for this role (API only)

CODE routers/badge_verification.py

GATE _assert_can_access_student (require_staff); require_admin for staff-awarded readiness badges

TABLES student_badges students

AWS none

Capability assessments

C R U D

1 endpoint · API only

C/U POST /api/mentor/students/{student_id}/assessments

record or correct scores API ONLY

SCREENS none for this role (API only)

CODE routers/badge_verification.py

GATE _assert_can_access_student (require_staff)

TABLES capability_assessments

AWS none

Skill claims (legacy queue, no screen)

€ R U Ð

3 endpoints · API only

- R** **GET** `/api/mentor/skill-claims/pending`
list pending skill claims **API ONLY**
- R** **GET** `/api/mentor/skill-claims/reviewed`
list reviewed skill claims **API ONLY**
- U** **POST** `/api/mentor/skill-claims/{claim_id}/review`
review skill claim **API ONLY**

SCREENS none for this role (API only)

CODE `routers/mentor.py`

GATE `require_mentor · require_mentor + _assert_can_access_student`

TABLES `skill_claims` `skills` `student_skills` `students` `uploads` `users`

AWS none

Skill Verifications: badge claims

€ R U Ð

4 endpoints · `/mentor/verifications`

- R** **GET** `/api/mentor/badge-evidence/pending`
list pending claims
- R** **GET** `/api/mentor/badge-evidence/reviewed`
list recently reviewed claims
- R** **GET** `/api/mentor/badge-evidence/{evidence_id}/file`
download claim certificate
- U** **POST** `/api/mentor/badge-evidence/{evidence_id}/review`
decide badge claim

SCREENS `/mentor/verifications`

CODE `routers/badge_verification.py`

GATE `require_capability(mentor.verifications) + _assert_can_access_student ·`
`require_capability(mentor.verifications) + mentor-group filter / scope_filter · capability`
`mentor.verifications`

TABLES `badge_evidence` `mail_logs` `student_badges` `students` `uploads` `users`

AWS EFS /data volume · Amazon SES mail

Skill Verifications: documents

€ R U ⌵

3 endpoints · /mentor/verifications

- R GET /api/mentor/uploads/pending
list documents to review
- R GET /api/mentor/uploads/{upload_id}/file
download student document
- U POST /api/mentor/uploads/{upload_id}/review
verify or reject document

SCREENS /mentor/verifications

CODE routers/mentor.py

GATE require_mentor · require_mentor + _assert_can_access_student

TABLES badge_evidence students uploads users

AWS EFS /data volume

Student badges & growth

€ R U ⌵

3 endpoints · API only

- R GET /api/mentor/students/{student_id}/badges
view student badge board API ONLY
- R GET /api/mentor/students/{student_id}/growth
view student growth API ONLY
- R GET /api/mentor/students/{student_id}/skill-profile
view consolidated skill profile API ONLY

SCREENS none for this role (API only)

CODE routers/badge_verification.py

GATE _assert_can_access_student (require_staff)

TABLES approved_certifications badge_evidence capability_assessments student_badges students
users

AWS none

Interview records



6 endpoints · API only

- R** **GET** `/api/mentor/interviews`
list all interviews **API ONLY**
- R** **GET** `/api/mentor/students/{student_id}/interviews`
list a student's interviews **API ONLY**
- R** **GET** `/api/mentor/students/{student_id}/interviews/progress`
view a student's interview trend **API ONLY**
- R** **GET** `/api/mentor/students/{student_id}/interviews/{session_id}`
open a student's interview **API ONLY**
- R** **GET** `/api/mentor/students/{student_id}/interviews/{session_id}/report`
view a student's scorecard **API ONLY**
- R** **GET** `/api/mentor/students/{student_id}/interviews/{session_id}/transcript`
view a student's transcript **API ONLY**

SCREENS none for this role (API only)

CODE `routers/interview_records.py`

GATE `_assert_can_access_student (require_staff + rule 2 mentor-group check) ·
_assert_can_access_student (require_staff + rule 2 mentor-group check) +
_session_of_student_or_404 · _assert_can_access_student (require_staff + rule 2 mentor-group
check) + _session_of_student_or_404; require_admin (via _may_see_raw_response) only decides
whether raw_response is included · require_mentor; _records_query narrows a MENTOR to
Student.mentor_id = session mentorId (no group = nobody)`

TABLES `interview_evaluations` `interview_score_summaries` `interview_sessions` `interview_turns`
`redesign_audit_events` `redesign_outbox_events` `students` `users`

AWS none

Mock interview consent



1 endpoint · API only

- R** **GET** `/api/interview/consent`
read my interview consent **API ONLY**

SCREENS none for this role (API only)

CODE `routers/interview_records.py`GATE `get_current_session (no role check)`TABLES `interview_consent`

AWS none

Focus log

2 endpoints · API only

€

R

U

Đ

R

GET

/api/mentor/students/{student_id}/focus
list student focus sessions API ONLY

U

POST

/api/mentor/focus/{session_id}/confirm
confirm focus session API ONLY

SCREENSnone for this role (API only)

CODErouters/mentor.py

GATE_assert_can_access_student (policies.assert_student_scope → require_staff) · require_mentor
+ _assert_can_access_student

TABLESlab_sessionsstudents

AWSnone

Mentee list (notebook picker)

1 endpoint · /mentor/notebook

€

R

U

Đ

R

GET

/api/v1/mentor/mentees
list my mentees

SCREENS/mentor/notebook

CODErouters/redesign.py

GATErequire_staff

TABLESredesign_tenant_membershipsstudentsusers

AWSnone

Mentee Log



4 endpoints · /mentor/mentees

- C** **POST** /api/mentor/students/{student_id}/notes
add meeting note
- R** **GET** /api/mentor/mentees
list my mentees
- R** **GET** /api/mentor/students/{student_id}/notes
list meeting notes
- D** **DELETE** /api/mentor/students/{student_id}/notes/{note_id}
retract meeting note

SCREENS /mentor/mentees

CODE routers/mentor.py

GATE require_capability(mentor.mentees) · require_capability(mentor.mentees) +
_assert_can_access_student · require_capability(mentor.mentees) +
_assert_can_access_student(allow_handover=True) · capability mentor.mentees

TABLES capability_grants mentor_notes students users

AWS none

Mentor notebook

C R U D

8 endpoints · /mentor/notebook

- C** **POST** /api/v1/mentor/notebook/entries/{entry_id}/attachments
register entry attachment **API ONLY**
- C** **POST** /api/v1/mentor/notebook/students/{student_id}/actions
add follow-up action **API ONLY**
- C** **POST** /api/v1/mentor/notebook/students/{student_id}/entries
write notebook entry
- R** **GET** /api/v1/mentor/notebook/students/{student_id}/actions
list notebook actions **API ONLY**
- R** **GET** /api/v1/mentor/notebook/students/{student_id}/entries
list notebook entries
- U** **PATCH** /api/v1/mentor/notebook/entries/{entry_id}
edit notebook entry **API ONLY**
- U** **POST** /api/v1/mentor/notebook/entries/{entry_id}/publish
publish entry to student
- D** **POST** /api/v1/mentor/notebook/entries/{entry_id}/archive
remove notebook entry

SCREENS /mentor/notebook

CODE routers/redesign.py

GATE require_staff + require_capability(mentor.notebook) + assert_student_scope · require_staff + require_capability(mentor.notebook) + assert_student_scope (+ expected_version optimistic lock) · require_staff + require_capability(mentor.notebook) + assert_student_scope + role == MENTOR · require_staff + require_capability(mentor.notebook) + assert_student_scope + role == MENTOR with mentorId · capability mentor.notebook

Leave 5 features

TABLES capability_grants redesign_api_idempotency_keys redesign_audit_events

College leave calendar

€ R U D

1 endpoint · API only

- R** **GET** /api/leaves/calendar
view my college calendar **API ONLY**

SCREENS none for this role (API only)

CODE routers/leave_policy.py

GATE get_current_session (caller's own college only)

TABLES academic_calendar cohorts departments students users

AWS none

Leave alternate arrangements



4 endpoints · /mentor/leave

- R** **GET** /api/leaves/alternate/mine
list requests naming me
- R** **GET** /api/leaves/{leave_id}/alternate
view alternate table **API ONLY**
- U** **POST** /api/leaves/{leave_id}/alternate/accept
accept covering classes
- U** **POST** /api/leaves/{leave_id}/alternate/assign
name covering colleague **API ONLY**

SCREENS /mentor/leave

CODE routers/leave_alternate.py

GATE get_current_session only (any signed-in account; only MENTOR/ADMIN accounts can be named, so others get an empty list) · get_current_session; applicant OR leave._assert_can_decide (require_mentor, then ADMIN or has_capability(admin.leave_approvals)) · get_current_session; applicant only (requester_user_id = caller, else flattened 404); request must be SUBMITTED/FIRST_APPROVED · get_current_session; caller must be the account named on a row (else flattened 404); request must be SUBMITTED/FIRST_APPROVED/APPROVED

TABLES capability_grants leave_requests users

AWS none

Leave attachments

C R U D

4 endpoints · /mentor/leave

- C** **POST** /api/leaves/{leave_id}/attachments
attach document to leave
- R** **GET** /api/leaves/{leave_id}/attachments
list leave attachments
- R** **GET** /api/leaves/{leave_id}/attachments/{attachment_id}/file
download leave attachment
- D** **DELETE** /api/leaves/{leave_id}/attachments/{attachment_id}
remove leave attachment

SCREENS /mentor/leave

CODE routers/leave_attachments.py

GATE get_current_session; applicant (requester_user_id = caller) OR leave._assert_can_decide (require_mentor + ADMIN, or has_capability(admin.leave_approvals)); every refusal flattened to 404 · get_current_session; applicant (requester_user_id = caller) OR leave._assert_can_decide (require_mentor + ADMIN, or has_capability(admin.leave_approvals)); every refusal flattened to 404; then _may_delete (the uploader, or role ADMIN for any file)

TABLES archived_documents capability_grants leave_attachments leave_requests users

AWS EFS /data volume · S3 documents archive

Leave policy & allowances

C R U D

2 endpoints · /mentor/leave

- R** **GET** /api/leaves/balances
view my leave allowances
- R** **GET** /api/leaves/{leave_id}/balance
view applicant's allowances **API ONLY**

SCREENS /mentor/leave

CODE routers/leave_policy.py

GATE get_current_session (own user id only) · requester themselves, else _assert_can_decide (require_mentor; ADMIN, or has_capability(admin.leave_approvals); other refusals flattened to 404)

TABLES capability_grants leave_balances leave_requests

AWS none

My own leave (apply / withdraw)

C R U D

4 endpoints · /mentor/leave

- C** **POST** /api/leaves
apply for leave
- R** **GET** /api/leaves/mine
list my leave
- R** **GET** /api/leaves/{leave_id}/paper.pdf
download leave PDF
- U** **POST** /api/leaves/{leave_id}/cancel
withdraw my leave

SCREENS /mentor/leave 2 features

CODE routers/leave.py routers/leave_paper.py

Staff signature

C R U D

4 endpoints · /mentor/signature

- R** **GET** /api/staff/signature
check my signature
- R** **GET** /api/staff/signature/image
preview my signature
- U/C** **PUT** /api/staff/signature
upload or replace signature
- D** **DELETE** /api/staff/signature
remove my signature

SCREENS /mentor/signature /account /mentor/leave

CODE routers/signature.py

GATE require_mentor (role MENTOR or ADMIN); always the caller's own row

TABLES archived_documents staff_signatures

AWS EFS /data volume · S3 documents archive

Upskilling shelf



4 endpoints · /mentor/upskilling

- C** **POST** /api/staff/upskilling
upload my certificate
- R** **GET** /api/staff/upskilling
list my certificates
- R** **GET** /api/staff/upskilling/{cert_id}/file
download my certificate
- D** **DELETE** /api/staff/upskilling/{cert_id}
remove my certificate

SCREENS /mentor/upskilling

CODE routers/staff_upskilling.py

GATE require_capability(mentor.upskilling) + cert.user_id == session.userId (404 otherwise) ·
require_capability(mentor.upskilling) + per-user VolumeQuota ·
require_capability(mentor.upskilling), rows filtered to session.userId · capability
mentor.upskilling

TABLES archived_documents staff_upskilling_certs

AWS EFS /data volume · S3 documents archive

REEP Agent

C R U D A

7 endpoints · /mentor/agent

C

POST

/api/agent/feedback

rate an answer

R

GET

/api/agent/history

read my chat history

R

GET

/api/agent/runs

list my agent runs API ONLY

D

DELETE

/api/agent/conversation

clear my conversation

A

POST

/api/agent/ask

ask the agent

A

POST

/api/agent/chat

chat with assistant API ONLY

A

POST

/api/agent/chat/stream

stream assistant reply API ONLY

SCREENS

/mentor/agent

(agent dock on every screen)

CODE

routers/agent.py

GATE

_require_agent_access: require_capability(mentor.agent) for MENTOR/ADMIN; llm_rate_limited · get_current_session (caller's own runs only) · get_current_session (caller's own thread only) · get_current_session + caller must own the agent run (404 otherwise) · capability mentor.agent

TABLES

agent_runs assistant_feedback conversations feature_overrides knowledge_chunks knowledge_documents messages student_profiles students

AWS

Embeddings (pgvector KB) · LLM (Bedrock / OpenAI-compatible)

Imports & analytics 1 feature

Student alerts

€ R U D

2 endpoints · API only

- R **GET** `/api/mentor/alerts`
list student alerts (API ONLY)
- U **POST** `/api/mentor/alerts/{alert_id}/resolve`
resolve alert (API ONLY)

SCREENS none for this role (API only)

CODE `routers/mentor.py`

GATE `require_mentor · require_mentor + _assert_can_access_student`

TABLES `alerts` `students` `users`

AWS none

Voice platform 1 feature

Voice platform call sessions

€ R U D

2 endpoints · API only

- R **GET** `/api/platform/calls`
list call sessions (API ONLY)
- R **GET** `/api/platform/calls/{session_id}`
view call detail (API ONLY)

SCREENS none for this role (API only)

CODE `voice_platform/api/calls.py`

GATE `get_current_session (any signed-in session); role = ADMIN lists every call, every other role is filtered to user_id = session.userId · get_current_session + _can_read (owner | role ADMIN | MENTOR via _assert_can_access_student on the linked interview's student); presigned recording URL only if has_capability(admin.interview_audio) · capability admin.interview_audio`

TABLES `capability_grants` `interview_sessions` `mentors` `platform_call_sessions`
`platform_recording_policies` `students`

AWS S3 call recordings

Granted by the Main Admin 17 capabilities

The Main Admin can hand any `admin.*` console capability to a named faculty account in **Who can do what**, with a reason and an expiry. The holder then uses the Main Admin's own endpoints, and rule 2 still limits anything about one student to their mentees.

admin.analytics

C R U D

12 endpoints · /admin/analytics

OPENS Analytics alert rules, Assign faculty, Batches & seating, Charts & numbers (analytics), Placement criteria

SCREENS /admin/analytics

admin.catalogue

C R U D

13 endpoints · /admin/catalogue

OPENS Certification & badge catalogue, Course catalogue & stage rules

SCREENS /admin/catalogue

admin.exports

€ R U D

4 endpoints · /admin/exports

OPENS Download reports

SCREENS /admin/exports

admin.governance

C R U D

19 endpoints · API only

OPENS College admins, Student feature switches, Who can do what (Governance)

SCREENS none (API only)

NOTE API only for a faculty deputy: both Governance screens are `roleGuard('ADMIN')`

admin.imports

C R U D

5 endpoints · /admin/imports

OPENS Upload spreadsheets (imports)

SCREENS /admin/imports

admin.institution

C R U D

22 endpoints · /admin/setup

OPENS Batches & seating, College structure, Colleges

SCREENS /admin/setup /admin/colleges /admin/institution

admin.interview_audio

€ R U D

2 endpoints · API only

OPENS Interview records

SCREENS none (API only)

admin.interview_questions

C R U D

10 endpoints · /admin/interview-questions

OPENS Interview questions, Interview tracks

SCREENS /admin/interview-questions

admin.interviews

C R U D

7 endpoints · /admin/interviews

OPENS Daily interview cap reset, Interview policy, Interview records

SCREENS /admin/interviews

admin.jobs

C R U D

4 endpoints · /admin/jobs

OPENS Job postings

SCREENS /admin/jobs

admin.leave_approvals

C R U D

10 endpoints · /admin/leave-approvals

OPENS Leave attachments, Leave requests (approve)

SCREENS /admin/leave-approvals

NOTE as a delegate: decide anybody's request and read its paper, alternates, allowance and attachments (their own request is in the lane above)

admin.mentors

€ R U D

4 endpoints · /admin/faculty

OPENS Assign faculty, Faculty designation & department

SCREENS /admin/faculty /admin/faculty/new /admin/mentors

NOTE the Faculty screen opens, but listing and adding faculty stay `require_admin`; the holder reaches designation & department and faculty assignment

admin.placement

€ R U D

1 endpoint · /admin/placement

OPENS Placement & offers

SCREENS /admin/placement

admin.registrations

C R U D

9 endpoints · /admin/registrations

OPENS Auto-approve rules, New applications (review queue)

SCREENS /admin/registrations

admin.student_records

€ R U D

1 endpoint · /admin/students/:id

OPENS Student 360 record

SCREENS /admin/students/:id

admin.students

€ R U D

9 endpoints · /admin/students

OPENS Batch promotion & graduation, Students & batches

SCREENS /admin/students

admin.swoc

C R U D

5 endpoints · /admin/swoc

OPENS SWOC notes

SCREENS /admin/swoc

ALUMNI

A graduate: their alumni profile, the jobs sheet, and what every signed-in account can do.

GATE `require_alumni` — no Student or Mentor row, no staff scope.

11 features **28** endpoints features that **4** create **11** read **3** update **2** delete **1** action

Programme (ledger, English, milestones) 1 feature

Login streak

1 endpoint · API only

R

GET

/api/student/streak

view my login streak

API ONLY

SCREENS none for this role (API only)

CODE `routers/student.py`

GATE `get_current_session` only (no `_require_student`; keyed on session `userId`)

TABLES `login_days`

AWS none

Interviews 1 feature

Mock interview consent

1 endpoint · API only

R

GET

/api/interview/consent

read my interview consent

API ONLY

SCREENS none for this role (API only)

CODE `routers/interview_records.py`

GATE `get_current_session` (no role check)

TABLES `interview_consent`s

AWS none

College leave calendar

€

R

U

Ð

1 endpoint · API only

R

GET

/api/leaves/calendar

view my college calendar

API ONLY

SCREENS none for this role (API only)

CODE routers/leave_policy.py

GATE get_current_session (caller's own college only)

TABLES academic_calendar cohorts departments students users

AWS none

Leave alternate arrangements

€

R

U

Ð

2 endpoints · API only

R

GET

/api/leaves/{leave_id}/alternate

view alternate table

API ONLY

U

POST

/api/leaves/{leave_id}/alternate/assign

name covering colleague

API ONLY

SCREENS none for this role (API only)

CODE routers/leave_alternate.py

GATE get_current_session; applicant OR leave._assert_can_decide (require_mentor, then ADMIN or has_capability(admin.leave_approvals)) · get_current_session; applicant only (requester_user_id = caller, else flattened 404); request must be SUBMITTED/FIRST_APPROVED

TABLES capability_grants leave_requests users

AWS none

Leave attachments

C R U D

4 endpoints · API only

- C** **POST** `/api/leaves/{leave_id}/attachments`
attach document to leave **API ONLY**
- R** **GET** `/api/leaves/{leave_id}/attachments`
list leave attachments **API ONLY**
- R** **GET** `/api/leaves/{leave_id}/attachments/{attachment_id}/file`
download leave attachment **API ONLY**
- D** **DELETE** `/api/leaves/{leave_id}/attachments/{attachment_id}`
remove leave attachment **API ONLY**

SCREENS none for this role (API only)

CODE `routers/leave_attachments.py`

GATE `get_current_session; applicant (requester_user_id == caller) OR leave._assert_can_decide (require_mentor + ADMIN, or has_capability(admin.leave_approvals)); every refusal flattened to 404 · get_current_session; applicant (requester_user_id == caller) OR leave._assert_can_decide (require_mentor + ADMIN, or has_capability(admin.leave_approvals)); every refusal flattened to 404; then _may_delete (the uploader, or role ADMIN for any file)`

TABLES `archived_documents` `capability_grants` `leave_attachments` `leave_requests` `users`

AWS EFS /data volume · S3 documents archive

Leave policy & allowances

C R U D

2 endpoints · API only

- R** **GET** `/api/leaves/balances`
view my leave allowances **API ONLY**
- R** **GET** `/api/leaves/{leave_id}/balance`
view applicant's allowances **API ONLY**

SCREENS none for this role (API only)

CODE `routers/leave_policy.py`

GATE `get_current_session (own user id only) · requester themselves, else _assert_can_decide (require_mentor; ADMIN, or has_capability(admin.leave_approvals); other refusals flattened to 404)`

TABLES `capability_grants` `leave_balances` `leave_requests`

AWS none

My own leave (apply / withdraw)

C R U D

4 endpoints · API only

- C** **POST** `/api/leaves`
apply for leave **API ONLY**
- R** **GET** `/api/leaves/mine`
list my leave **API ONLY**
- R** **GET** `/api/leaves/{leave_id}/paper.pdf`
download leave PDF **API ONLY**
- U** **POST** `/api/leaves/{leave_id}/cancel`
withdraw my leave **API ONLY**

SCREENS none for this role (API only)

CODE `routers/leave.py` `routers/leave_paper.py`

GATE `get_current_session (+ leave_policy.submit_refusal balance/overlap checks)` · `get_current_session (requester_user_id = caller)` · `get_current_session (requester_user_id = caller, else 404)` · `requester themselves, else _assert_can_decide (require_mentor; ADMIN, or has_capability(admin.leave_approvals); other refusals flattened to 404)`

TABLES `academic_calendar` `capability_grants` `leave_balances` `leave_requests` `mail_logs`
`staff_signatures` `students` `users`

AWS EFS /data volume · PDF render (in process) · Amazon SES mail

Alumni 2 features

Alumni jobs sheet

C R U D

1 endpoint · `/alumni/jobs`

- R** **GET** `/api/alumni/jobs`
list open job postings

SCREENS `/alumni/jobs`

CODE `routers/alumni.py`

GATE `require_alumni (role ALUMNI)`

TABLES `academic_specializations` `alumni_profiles` `cohorts` `jobs` `students`

AWS none

Alumni profile

C R U D

3 endpoints · /alumni

- C/U

POST

/api/alumni/profile

create (first login) or edit profile
- R

GET

/api/alumni/profile

read my alumni profile
- R

GET

/api/alumni/profile/resume

download my resume

- SCREENS

/alumni
- CODE

routers/alumni.py
- GATE

require_alumni (role ALUMNI)
- TABLES

alumni_profiles

archived_documents
- AWS

EFS /data volume · S3 documents archive

REEP Agent

C R U D A

7 endpoints · (agent dock on every screen)

C

POST

/api/agent/feedback

rate an answer

R

GET

/api/agent/history

read my chat history

R

GET

/api/agent/runs

list my agent runs

API ONLY

D

DELETE

/api/agent/conversation

clear my conversation

A

POST

/api/agent/ask

ask the agent

A

POST

/api/agent/chat

chat with assistant

API ONLY

A

POST

/api/agent/chat/stream

stream assistant reply

API ONLY

SCREENS

(agent dock on every screen)

CODE

routers/agent.py

GATE

_require_agent_access: require_capability(mentor.agent) for MENTOR/ADMIN;

require_feature(student.agent) for STUDENT; llm_rate_limited · get_current_session (caller's own runs only) · get_current_session (caller's own thread only) · get_current_session + caller must own the agent run (404 otherwise)

TABLES

agent_runs

assistant_feedback

conversations

feature_overrides

knowledge_chunks

knowledge_documents

messages

student_profiles

students

AWS

Embeddings (pgvector KB) · LLM (Bedrock / OpenAI-compatible)

Voice platform call sessions

2 endpoints · API only

€

R

U

Ð

R

GET

/api/platform/calls

list call sessions

API ONLY

R

GET

/api/platform/calls/{session_id}

view call detail

API ONLY

SCREENS

none for this role (API only)

CODE

voice_platform/api/calls.py

GATE

get_current_session (any signed-in session); role = ADMIN lists every call, every other role is filtered to user_id = session.userId · get_current_session + _can_read (owner | role ADMIN | MENTOR via _assert_can_access_student on the linked interview's student); presigned recording URL only if has_capability(admin.interview_audio)

TABLES

capability_grants interview_sessions mentors platform_call_sessions platform_recording_policies students

AWS

S3 call recordings

MAIN ADMIN (role ADMIN, the placement office)

The placement office, one account (role ADMIN): the whole console.

GATE `require_admin` (the one console gate) or `require_capability(admin.*)` held by baseline. Not a faculty member: mentor.mentees / notebook / verifications / upskilling are outside its baseline.

62 features 245 endpoints features that 26 create 56 read 40 update
22 delete 3 action

Identity & sign-in 3 features

Account lifecycle

3 endpoints · /admin/faculty

U

POST

/api/admin/users/{user_id}/disable

disable account

U

POST

/api/admin/users/{user_id}/enable

re-enable account

U

POST

/api/admin/users/{user_id}/sign-out-everywhere

sign out every device

SCREENS

/admin/faculty /admin/students/:id

CODE

routers/admin_faculty.py

GATE

require_admin

TABLES

auth_tokens mentor_assignments mentors redesign_audit_events redesign_outbox_events

students users

AWS

none

Faculty (accounts)

C R U D

4 endpoints · /admin/faculty/new

- C** **POST** /api/admin/faculty
add faculty member
- C** **POST** /api/admin/users/{user_id}/activation-link
mint activation link
- R** **GET** /api/admin/faculty
list faculty accounts
- U** **PATCH** /api/admin/faculty/{user_id}
edit faculty member

SCREENS /admin/faculty/new /admin/faculty /admin/colleges

CODE routers/admin.py routers/admin_faculty.py

GATE require_admin

TABLES auth_tokens colleges departments mail_logs redesign_audit_events redesign_outbox_events
users

AWS Amazon SES mail

ALSO remove, restore and delete a faculty account: see [Remove / delete people](#)

Faculty designation & department

C R U D

1 endpoint · API only

- U** **PATCH** /api/admin/users/{user_id}/institutional-identity
set designation and department **API ONLY**

SCREENS none for this role (API only)

CODE routers/admin.py

GATE require_capability(admin.mentors) · capability admin.mentors

TABLES users

AWS none

Auto-approve rules

C R U D

4 endpoints · /admin/registrations

- C POST /api/register/rules
create seating rule
- R GET /api/register/rules
list seating rules
- U PATCH /api/register/rules/{rule_id}
edit seating rule
- D DELETE /api/register/rules/{rule_id}
delete seating rule

SCREENS /admin/registrations

CODE routers/registration.py

GATE require_capability(admin.registrations) + _assert_rule_reachable (scope) ·
require_capability(admin.registrations) + _assert_rule_reachable before and after (scope) ·
require_capability(admin.registrations) + scope_filter (X-Reep-Scope header) · capability
admin.registrations

TABLES capability_grants cohorts redesign_audit_events redesign_outbox_events
registration_rules registrations

AWS none

New applications (review queue)

€ R U ⓧ

5 endpoints · /admin/registrations

- R** **GET** /api/register/pending
list applications
- R** **GET** /api/register/{registration_id}/documents/{kind}/file
download applicant document
- U** **POST** /api/register/{registration_id}/decision
approve or reject application
- U** **POST** /api/register/{registration_id}/hold
hold application
- U** **POST** /api/register/{registration_id}/reopen
reopen application

SCREENS /admin/registrations /admin

CODE routers/registration.py

GATE require_capability(admin.registrations) + _assert_reachable (scope) ·
require_capability(admin.registrations) + scope_filter (X-Reep-Scope header) · capability
admin.registrations

TABLES academic_courses academic_specializations archived_documents auth_tokens
capability_grants cohorts colleges departments mail_logs redesign_audit_events
redesign_outbox_events registration_documents registration_rules registrations
student_profiles students uploads users

AWS EFS /data volume · Amazon SES mail

Mail & notifications 1 feature

Email delivery

€ R ⓧ D

3 endpoints · /admin/mail

- R** **GET** /api/admin/mail-log
list sent mail log
- R** **GET** /api/admin/mail-log/suppression
check address suppression
- D** **DELETE** /api/admin/mail-log/suppression
lift SES suppression

SCREENS /admin/mail

CODE routers/admin_mail.py

GATE require_admin (via _office dependency)

TABLES mail_logs redesign_audit_events redesign_outbox_events

AWS Amazon SES mail

Student records & roster 3 features

Batch promotion & graduation



4 endpoints · /admin/students

R **GET** /api/admin/cohorts/{cohort_id}/promotion-history

list batch semester history

U **POST** /api/admin/cohorts/{cohort_id}/graduate

graduate a batch

U **POST** /api/admin/cohorts/{cohort_id}/promote

promote a batch

U **POST** /api/admin/cohorts/{cohort_id}/ungraduate

reverse a graduation

SCREENS /admin/students

CODE routers/admin_promotion.py

GATE require_capability(admin.students) + _cohort_or_404 + assert_batch_within_reach (all-or-nothing scope over the batch) · capability admin.students

TABLES academic_courses academic_specializations alumni_profiles capability_grants cohorts redesign_audit_events redesign_outbox_events semester_results stage_rules student_semester_history students users

AWS none

Student 360 record



1 endpoint · /admin/students/:id

R **GET** /api/admin/students/{student_id}/360

view full student record

SCREENS /admin/students/:id

CODE routers/admin_student_360.py

GATE require_capability(admin.student_records, target=ancestry_of_student) + _assert_can_access_student · capability admin.student_records

TABLES academic_courses academic_specializations attendance_records badge_evidence certification_progress cohorts departments interview_evaluations interview_sessions login_events mentor_assignments mentors placement_criteria redesign_audit_events resume_profiles semester_results student_badges student_profiles student_semester_history students time_ledger_days uploads users

AWS none

Students & batches

€ R U D

5 endpoints · /admin/students

- R** **GET** /api/admin/students
list student roster
- U** **POST** /api/admin/cohorts/{cohort_id}/students/bulk
bulk update batch
- U** **POST** /api/admin/students/bulk
file students under department **API ONLY**
- U** **PATCH** /api/admin/students/{student_id}
edit one student
- D** **DELETE** /api/admin/cohorts/{cohort_id}
delete empty batch

SCREENS /admin/students /admin/governance/features /admin/mentors /admin/students/:id

CODE routers/admin_students.py

GATE require_capability(admin.students) · require_capability(admin.students) +
assert_batch_within_reach (all-or-nothing scope) · require_capability(admin.students) +
require_capability(admin.students, target=ancestry_of_student) ·
require_capability(admin.students) + scope_filter(admin.students) · capability admin.students

TABLES academic_courses academic_specializations capability_grants cohorts departments
mentor_assignments mentors redesign_audit_events redesign_outbox_events student_profiles
students users

AWS none

ALSO remove, restore and delete a student: see **Remove / delete people**; DELETE here is an EMPTY batch only

Programme (ledger, English, milestones) 1 feature

Login streak

€ R U D

1 endpoint · API only

- R** **GET** /api/student/streak
view my login streak **API ONLY**

SCREENS none for this role (API only)

CODE routers/student.py

GATE get_current_session only (no _require_student; keyed on session userId)

TABLES login_days

AWS none

Badge awards

C R U D

2 endpoints · API only

C

POST

/api/mentor/students/{student_id}/badges/{code}/award

manually award badge

API ONLY

D

POST

/api/mentor/students/{student_id}/badges/{code}/revoke

revoke earned badge

API ONLY

SCREENS

none for this role (API only)

CODE

routes/badge_verification.py

GATE

_assert_can_access_student (require_staff); require_admin for staff-awarded readiness badges · require_admin

TABLES

redesign_audit_events redesign_outbox_events student_badges students

AWS

none

Capability assessments

C R U D

1 endpoint · API only

C/U

POST

/api/mentor/students/{student_id}/assessments

record or correct scores

API ONLY

SCREENS

none for this role (API only)

CODE

routes/badge_verification.py

GATE

_assert_can_access_student (require_staff)

TABLES

capability_assessments

AWS

none

Certification & badge catalogue

C R U D

6 endpoints · /admin/catalogue

- C** **POST** /api/admin/approved-certifications
add approved certification
- R** **GET** /api/admin/approved-certifications
list approved certifications
- R** **GET** /api/admin/badge-catalogue
list badge catalogue
- R** **GET** /api/admin/catalogue/badges
list badges for course (API ONLY)
- U** **PATCH** /api/admin/approved-certifications/{cert_id}
edit approved certification
- U** **PUT** /api/admin/catalogue/badges
switch badge for course (API ONLY)

SCREENS /admin/catalogue

CODE routers/admin_catalogue.py routers/console.py

GATE require_capability(admin.catalogue) · require_capability(admin.catalogue) +
_assert_certification_within_reach (old scope) + _resolve_certification_scope (new scope) ·
require_capability(admin.catalogue) + _course_within_reach ·
require_capability(admin.catalogue) + _resolve_certification_scope (scoped holder must name a
college/course in reach) · require_capability(admin.catalogue) +
scope_filter(admin.catalogue) · capability admin.catalogue

TABLES academic_courses approved_certifications badge_course_map badge_evidence colleges
departments redesign_audit_events redesign_outbox_events

AWS none

Skill claims (legacy queue, no screen)

€ R U D

3 endpoints · API only

- R** **GET** /api/mentor/skill-claims/pending
list pending skill claims (API ONLY)
- R** **GET** /api/mentor/skill-claims/reviewed
list reviewed skill claims (API ONLY)
- U** **POST** /api/mentor/skill-claims/{claim_id}/review
review skill claim (API ONLY)

SCREENS none for this role (API only)

CODE routers/mentor.py

GATE require_mentor · require_mentor + _assert_can_access_student

TABLES skill_claims skills student_skills students uploads users

AWS none

Skill Verifications: documents

€ R U D

3 endpoints · /admin/students/:id

- R** **GET** /api/mentor/uploads/pending
list documents to review **API ONLY**
- R** **GET** /api/mentor/uploads/{upload_id}/file
download student document
- U** **POST** /api/mentor/uploads/{upload_id}/review
verify or reject document **API ONLY**

SCREENS /admin/students/:id

CODE routers/mentor.py

GATE require_mentor · require_mentor + _assert_can_access_student

TABLES badge_evidence students uploads users

AWS EFS /data volume

Student badges & growth

€ R U D

3 endpoints · /admin/students/:id

- R** **GET** /api/mentor/students/{student_id}/badges
view student badge board
- R** **GET** /api/mentor/students/{student_id}/growth
view student growth **API ONLY**
- R** **GET** /api/mentor/students/{student_id}/skill-profile
view consolidated skill profile **API ONLY**

SCREENS /admin/students/:id

CODE routers/badge_verification.py

GATE _assert_can_access_student (require_staff)

TABLES approved_certifications badge_evidence capability_assessments student_badges students
users

AWS none

Daily interview cap reset

C R U D

1 endpoint · /admin/students/:id

C **POST** /api/admin/students/{student_id}/interview-cap/reset
reset student's interview cap

SCREENS /admin/students/:id

CODE routers/interview_policy.py

GATE require_capability(admin.interviews, target=ancestry_of_student) + _assert_can_access_student · capability admin.interviews

TABLES capability_grants interview_cap_resets interview_policies interview_sessions mentors
redesign_audit_events redesign_outbox_events students

AWS none

Interview policy

C R U D

3 endpoints · /admin/interviews

R **GET** /api/admin/interview-policies/{college_id}
view college interview policy

U **PUT** /api/admin/interview-policies/{college_id}
set college interview policy

U **PUT** /api/admin/interview-policies/{college_id}/{course_id}
set course interview policy

SCREENS /admin/interviews

CODE routers/interview_policy.py

GATE require_capability(admin.interviews) + _assert_reach (college) ·
require_capability(admin.interviews) + _assert_reach (college-level reach) ·
require_capability(admin.interviews) + _assert_reach (course or its college) · capability
admin.interviews

TABLES academic_courses capability_grants colleges departments interview_policies
redesign_audit_events redesign_outbox_events users

AWS none

Interview questions

6 endpoints · /admin/interview-questions

- C

POST

/api/admin/interview-questions

add a question
- C

POST

/api/admin/interview-questions/bulk

bulk add questions
- R

GET

/api/admin/interview-questions

list track questions
- U

POST

/api/admin/interview-questions/reorder

reorder track questions
- U

PATCH

/api/admin/interview-questions/{question_id}

edit a question
- D

DELETE

/api/admin/interview-questions/{question_id}

delete a question

SCREENS /admin/interview-questions

CODE routers/interview_bank.py

GATE require_capability(admin.interview_questions) + _check_readable_track + college reach (scope_filter) · require_capability(admin.interview_questions) + _check_track (authorable tracks within reach) · require_capability(admin.interview_questions) + _require_question_reach (track's college/course/specialization) · capability admin.interview_questions

TABLES capability_grants interview_bank_questions interview_tracks redesign_audit_events redesign_outbox_events

AWS none

Interview records



11 endpoints · /admin/interviews

- R** **GET** /api/admin/interviews
list interview records
- R** **GET** /api/admin/interviews/export.csv
export interview scores CSV
- R** **GET** /api/admin/interviews/summary
show interview KPIs
- R** **GET** /api/mentor/interviews
list all interviews **API ONLY**
- R** **POST** /api/mentor/interviews/audio.zip
download recordings as zip
- R** **GET** /api/mentor/students/{student_id}/interviews
list a student's interviews
- R** **GET** /api/mentor/students/{student_id}/interviews/progress
view a student's interview trend **API ONLY**
- R** **GET** /api/mentor/students/{student_id}/interviews/{session_id}
open a student's interview **API ONLY**
- R** **GET** /api/mentor/students/{student_id}/interviews/{session_id}/audio
play interview recording
- R** **GET** /api/mentor/students/{student_id}/interviews/{session_id}/report
view a student's scorecard
- R** **GET** /api/mentor/students/{student_id}/interviews/{session_id}/transcript
view a student's transcript

SCREENS /admin/interviews /admin/exports /admin/students/:id

CODE routers/interview_records.py

GATE `_assert_can_access_student (require_staff + rule 2 mentor-group check) ·
_assert_can_access_student (require_staff + rule 2 mentor-group check) +
_session_of_student_or_404 · _assert_can_access_student (require_staff + rule 2 mentor-group
check) + _session_of_student_or_404; require_admin (via _may_see_raw_response) only decides
whether raw_response is included · _require_developer →
require_capability(admin.interview_audio) + _assert_can_access_student (require_staff + rule 2
mentor-group check) + _session_of_student_or_404 · _require_developer →
require_capability(admin.interview_audio); each id re-checked with _assert_can_access_student
· require_capability(admin.interviews) + scope_filter + carries_personal_columns;
record_export receipt · require_capability(admin.interviews) + scope_filter(admin.interviews)
· require_capability(admin.interviews) + scope_filter(admin.interviews); _records_query also
narrows a MENTOR to their own mentor group · require_mentor; _records_query narrows a MENTOR
to Student.mentor_id = session mentorId (no group = nobody) · capability
admin.interview_audio admin.interviews`

TABLES capability_grants export_events interview_evaluations interview_score_summaries
interview_sessions interview_turns redesign_audit_events redesign_outbox_events students

Interview tracks

C R U D

4 endpoints · /admin/interview-questions

- C** POST /api/admin/interview-questions/tracks
add interview track
- R** GET /api/admin/interview-questions/tracks
list interview tracks
- U** PATCH /api/admin/interview-questions/tracks/{track_id}
edit interview track
- D** DELETE /api/admin/interview-questions/tracks/{track_id}
delete interview track

SCREENS /admin/interview-questions /admin/catalogue /admin/institution /admin/setup

CODE routers/admin_interview_tracks.py

GATE require_capability(admin.interview_questions) + _require_reach (target = new track's spine) ·
require_capability(admin.interview_questions) + _require_reach (target = track's spine
ancestry) · require_capability(admin.interview_questions) + _require_reach on the row as it
stands and as it would be · require_capability(admin.interview_questions) +
scope_filter(admin.interview_questions) · capability admin.interview_questions

TABLES academic_courses academic_specializations colleges departments interview_bank_questions
interview_sessions interview_tracks redesign_audit_events redesign_outbox_events

AWS none

Mock interview



2 endpoints · (agent dock on every screen)

R **GET** `/api/interview/status`
check interview availability

A **WS** `/api/interview`
run live mock interview

SCREENS (agent dock on every screen)

CODE routers/interview.py

GATE `get_current_session; available only to role STUDENT or ADMIN (_is_rehearsal), any other role gets 200 available=false · get_ws_session cookie + Origin check; role STUDENT or ADMIN (_is_rehearsal) else close 1008; student.assistant feature switch (close 4016); live consent (4013); concurrency and daily/attempt caps (1013/4012/4015)`

TABLES archived_documents cohorts conversations feature_overrides interview_bank_questions interview_cap_resets interview_consents interview_evaluations interview_policies interview_score_summaries interview_sessions interview_tracks interview_turns messages

Mentoring & SWOC

4 features

Assign faculty



4 endpoints · /admin/mentors

R **GET** `/api/admin/mentor-load`
view faculty mentee load

R **GET** `/api/admin/students/{student_id}/mentor-history`
view mentor history

R **GET** `/api/admin/unassigned-students`
list unassigned students

U **POST** `/api/admin/students/{student_id}/mentor`
assign or release faculty

SCREENS /admin/mentors /admin/students /admin/faculty /admin/analytics /admin

CODE routers/admin_mentoring.py

GATE `require_capability(admin.analytics) + scope_filter · require_capability(admin.mentors) + require_capability(admin.mentors, target=ancestry_of_student) · require_capability(admin.mentors) + scope_filter · require_capability(admin.mentors) bare + target=ancestry_of_student + target=ancestry_of_user(faculty) + _assert_same_college · capability admin.analytics admin.mentors`

TABLES attendance_records capability_grants cohorts colleges departments mentor_assignments mentors redesign_audit_events redesign_outbox_events student_skills students time_ledger_cells time_ledger_days users

AWS none

Focus log

€ R U D

2 endpoints · API only

R **GET** `/api/mentor/students/{student_id}/focus`
list student focus sessions **API ONLY**

U **POST** `/api/mentor/focus/{session_id}/confirm`
confirm focus session **API ONLY**

SCREENS none for this role (API only)

CODE `routers/mentor.py`

GATE `_assert_can_access_student (policies.assert_student_scope → require_staff) · require_mentor`
`+ _assert_can_access_student`

TABLES `lab_sessions` `students`

AWS none

Mentee list (notebook picker)

€ R U D

1 endpoint · API only

R **GET** `/api/v1/mentor/mentees`
list my mentees **API ONLY**

SCREENS none for this role (API only)

CODE `routers/redesign.py`

GATE `require_staff`

TABLES `redesign_tenant_memberships` `students` `users`

AWS none

SWOC notes

C R U D

5 endpoints · /admin/swoc

C	POST	/api/admin/swoc/{student_id}	add SWOC line
R	GET	/api/admin/swoc	list SWOC boards
R	GET	/api/admin/swoc/{student_id}/history	view SWOC edit history
U	PATCH	/api/admin/swoc/entries/{entry_id}	edit SWOC line
D	DELETE	/api/admin/swoc/entries/{entry_id}	delete SWOC line

SCREENS /admin/swoc

CODE routers/swoc.py

GATE `require_capability(admin.swoc) + require_capability(admin.swoc, target=ancestry_of_student) ·`
`require_capability(admin.swoc) + require_capability(admin.swoc, target=ancestry_of_student) +`
`_assert_owns (author or Main Admin) · require_capability(admin.swoc) +`
`scope_filter(admin.swoc) · capability admin.swoc`

TABLES `academic_courses` `academic_specializations` `capability_grants` `cohorts` `interview_sessions`
`jobs` `mentors` `redesign_audit_events` `redesign_outbox_events` `student_skills` `students`
`swoc_entries` `swoc_entry_revisions` `users`

AWS none

College leave calendar

C R U D

4 endpoints · /admin/leave-approvals

- R GET /api/admin/leave-calendar/{college_id}
list college calendar
- R GET /api/leaves/calendar
view my college calendar API ONLY
- U/C PUT /api/admin/leave-calendar/{college_id}
add or change calendar day
- D DELETE /api/admin/leave-calendar/{college_id}/{day_id}
delete calendar day

SCREENS /admin/leave-approvals

CODE routers/leave_policy.py

GATE get_current_session (caller's own college only) · require_admin

TABLES academic_calendar cohorts colleges departments redesign_audit_events
redesign_outbox_events students users

AWS none

Leave alternate arrangements

€ R U ⌕

4 endpoints · /admin/leave-approvals

- R GET /api/leaves/alternate/mine
list requests naming me API ONLY
- R GET /api/leaves/{leave_id}/alternate
view alternate table
- U POST /api/leaves/{leave_id}/alternate/accept
accept covering classes API ONLY
- U POST /api/leaves/{leave_id}/alternate/assign
name covering colleague API ONLY

SCREENS /admin/leave-approvals

CODE routers/leave_alterate.py

GATE get_current_session only (any signed-in account; only MENTOR/ADMIN accounts can be named, so others get an empty list) · get_current_session; applicant OR leave._assert_can_decide (require_mentor, then ADMIN or has_capability(admin.leave_approvals)) · get_current_session; applicant only (requester_user_id == caller, else flattened 404); request must be SUBMITTED/FIRST_APPROVED · get_current_session; caller must be the account named on a row (else flattened 404); request must be SUBMITTED/FIRST_APPROVED/APPROVED

TABLES capability_grants leave_requests users

AWS none

Leave attachments



4 endpoints · /admin/leave-approvals

- C** **POST** /api/leaves/{leave_id}/attachments
attach document to leave **API ONLY**
- R** **GET** /api/leaves/{leave_id}/attachments
list leave attachments
- R** **GET** /api/leaves/{leave_id}/attachments/{attachment_id}/file
download leave attachment
- D** **DELETE** /api/leaves/{leave_id}/attachments/{attachment_id}
remove leave attachment **API ONLY**

SCREENS /admin/leave-approvals

CODE routers/leave_attachments.py

GATE
get_current_session; applicant (requester_user_id = caller) OR leave._assert_can_decide (require_mentor + ADMIN, or has_capability(admin.leave_approvals)); every refusal flattened to 404 · get_current_session; applicant (requester_user_id = caller) OR leave._assert_can_decide (require_mentor + ADMIN, or has_capability(admin.leave_approvals)); every refusal flattened to 404; then _may_delete (the uploader, or role ADMIN for any file)

TABLES archived_documents capability_grants leave_attachments leave_requests users

AWS EFS /data volume · S3 documents archive

Leave policy & allowances

C R U D

7 endpoints · /admin/leave-approvals

- C** **POST** /api/admin/leave-balances/bulk
set department allowances
- R** **GET** /api/admin/leave-balances
list leave allowances
- R** **GET** /api/admin/leave-policy
view leave policy sheet
- R** **GET** /api/leaves/balances
view my leave allowances **API ONLY**
- R** **GET** /api/leaves/{leave_id}/balance
view applicant's allowances
- U** **PUT** /api/admin/leave-balances
record one allowance
- D** **DELETE** /api/admin/leave-balances/{balance_id}
delete leave allowance

SCREENS /admin/leave-approvals

CODE routers/leave_policy.py

GATE get_current_session (own user id only) · requester themselves, else _assert_can_decide (require_mentor; ADMIN, or has_capability(admin.leave_approvals); other refusals flattened to 404) · require_admin

TABLES academic_calendar capability_grants cohorts colleges departments leave_balances leave_requests redesign_audit_events redesign_outbox_events students users

AWS none

Leave requests (approve)

€ R U D

3 endpoints · /admin/leave-approvals

- R** **GET** /api/leaves/history
list decided leave
- R** **GET** /api/leaves/pending
list pending leave
- U** **POST** /api/leaves/{leave_id}/decision
approve or reject leave

SCREENS /admin/leave-approvals /admin

CODE routers/leave.py

GATE _require_leave_approver: require_mentor + require_capability(admin.leave_approvals) ·
_require_leave_approver: require_mentor + require_capability(admin.leave_approvals) +
_assert_can_decide · capability admin.leave_approvals

TABLES capability_grants leave_requests mail_logs users

AWS Amazon SES mail

My own leave (apply / withdraw)

C R U D

4 endpoints · /admin/leave-approvals

- C** **POST** /api/leaves
apply for leave (API ONLY)
- R** **GET** /api/leaves/mine
list my leave (API ONLY)
- R** **GET** /api/leaves/{leave_id}/paper.pdf
download leave PDF
- U** **POST** /api/leaves/{leave_id}/cancel
withdraw my leave (API ONLY)

SCREENS /admin/leave-approvals

CODE routers/leave.py routers/leave_paper.py

GATE get_current_session (+ leave_policy.submit_refusal balance/overlap checks) ·
get_current_session (requester_user_id = caller) · get_current_session (requester_user_id =
caller, else 404) · requester themselves, else _assert_can_decide (require_mentor; ADMIN, or
has_capability(admin.leave_approvals); other refusals flattened to 404)

TABLES academic_calendar capability_grants leave_balances leave_requests mail_logs
staff_signatures students users

AWS EFS /data volume · PDF render (in process) · Amazon SES mail

Job postings

C R U D

4 endpoints · /admin/jobs

C

POST

/api/admin/jobs

publish job posting

R

GET

/api/admin/jobs

list job postings

U

POST

/api/admin/jobs/{job_id}/close

close job posting

D

DELETE

/api/admin/jobs/{job_id}

remove job posting

SCREENS

/admin/jobs

CODE

routers/console.py

GATE

require_capability(admin.jobs) · require_capability(admin.jobs) + scope_filter(admin.jobs) · capability admin.jobs

TABLES

academic_courses colleges job_applications jobs redesign_audit_events redesign_outbox_events

AWS

none

Placement & offers

€ R U Ð

3 endpoints · /admin/placement

R

GET

/api/admin/placement

read placement funnel

R

GET

/api/mentor/offers/pending

list offers awaiting approval

U

POST

/api/mentor/offers/{offer_id}/decision

approve or reject offer

SCREENS

/admin/placement /admin

CODE

routers/console.py routers/mentor.py

GATE

require_admin · require_capability(admin.placement) + scope_filter(admin.placement) · capability admin.placement

TABLES

academic_specializations cohorts job_applications placement_offers students users

AWS

none

Placement criteria

C R U D

3 endpoints · /admin/imports

- C** **POST** /api/admin/criteria
set placement criteria
- R** **GET** /api/admin/criteria
read placement criteria
- R** **GET** /api/admin/criteria/history
list criteria history

SCREENS /admin/imports /admin/jobs /admin/analytics

CODE routers/console.py

GATE require_capability(admin.analytics) · require_capability(admin.analytics) +
scope_filter(admin.analytics) · capability admin.analytics

TABLES academic_courses colleges departments placement_criteria redesign_audit_events
redesign_outbox_events users

AWS none

Staff self-service 1 feature

Staff signature

C R U D

4 endpoints · /mentor/signature

- R** **GET** /api/staff/signature
check my signature
- R** **GET** /api/staff/signature/image
preview my signature
- U/C** **PUT** /api/staff/signature
upload or replace signature
- D** **DELETE** /api/staff/signature
remove my signature

SCREENS /mentor/signature /account

CODE routers/signature.py

GATE require_mentor (role MENTOR or ADMIN); always the caller's own row

TABLES archived_documents staff_signatures

AWS EFS /data volume · S3 documents archive

REEP Agent

C R U D A

7 endpoints · /admin/agent

C

POST

/api/agent/feedback

rate an answer

R

GET

/api/agent/history

read my chat history

R

GET

/api/agent/runs

list my agent runs

API ONLY

D

DELETE

/api/agent/conversation

clear my conversation

A

POST

/api/agent/ask

ask the agent

A

POST

/api/agent/chat

chat with assistant

API ONLY

A

POST

/api/agent/chat/stream

stream assistant reply

API ONLY

SCREENS

/admin/agent

(agent dock on every screen)

CODE

routers/agent.py

GATE

`_require_agent_access: require_capability(mentor.agent) for MENTOR/ADMIN; llm_rate_limited · get_current_session (caller's own runs only) · get_current_session (caller's own thread only) · get_current_session + caller must own the agent run (404 otherwise) · capability mentor.agent`

TABLES

agent_runs assistant_feedback conversations feature_overrides knowledge_chunks knowledge_documents messages student_profiles students

AWS

Embeddings (pgvector KB) · LLM (Bedrock / OpenAI-compatible)

REEP Agent metrics

€ R U D

1 endpoint · API only

R

GET

/api/agent/metrics

view assistant metrics

API ONLY

SCREENS

none for this role (API only)

CODE

routers/agent.py

GATE

`get_current_session + inline role = ADMIN`

TABLES

agent_runs assistant_feedback messages

AWS

none

Batches & seating

4 endpoints · /admin/analytics

R

GET

/api/admin/cohorts

list batches

R

GET

/api/admin/cohorts/{cohort_id}/students

list students in batch

R

GET

/api/admin/students/unseated

list unseated students

U

PUT

/api/admin/students/{student_id}/cohort

seat or unseat student

SCREENS

/admin/analytics

/admin/students/:id

/admin/placement

/admin/mentors

/admin/interviews

/admin/institution

CODE

routers/admin.py

routers/console.py

GATE

require_capability(admin.analytics)

·

require_capability(admin.institution)

·

capability

admin.analytics

admin.institution

TABLES

academic_courses

academic_specializations

cohorts

students

users

AWS

none

College structure



16 endpoints · /admin/setup

POST /api/admin/academic-courses/{course-id}/academic-specializations

Colleges

C R U D

3 endpoints · /admin/setup

C **POST** /api/admin/colleges
add college

R **GET** /api/admin/colleges
list colleges

U **PATCH** /api/admin/colleges/{college-id}
edit college

SCREENS /admin/setup /admin/colleges /admin/institution /admin/faculty/new /admin/interviews

CODE routers/admin.py

GATE require_capability(admin.institution) · capability admin.institution

TABLES colleges departments

AWS none

ALSO delete a college: see **Delete a college**

Course catalogue & stage rules

C R U D

7 endpoints · /admin/catalogue

C **POST** /api/admin/catalogue/copy
copy catalogue to course

C **POST** /api/admin/catalogue/subjects/import
import subjects CSV

R **GET** /api/admin/catalogue
list courses and certifications

R **GET** /api/admin/catalogue/courses
list catalogue courses

R **GET** /api/admin/catalogue/stage-rules
list stage rules

U **PUT** /api/admin/catalogue/stage-rules
set stage rule

D **DELETE** /api/admin/catalogue/stage-rules/{rule-id}
delete stage rule

SCREENS /admin/catalogue /admin/jobs

CODE routers/admin_catalogue.py routers/console.py

GATE require_capability(admin.catalogue) · require_capability(admin.catalogue) +
_course_within_reach · require_capability(admin.catalogue) + _course_within_reach (target =
course ancestry) · require_capability(admin.catalogue) + _course_within_reach on both
from_course and to_course · require_capability(admin.catalogue) + scope_filter must be
programme-wide (a scoped grant is refused) · require_capability(admin.catalogue) +
scope_filter(admin.catalogue) · capability admin.catalogue

Imports & analytics

TABLES academic_courses approved_certifications badge_course_map certifications colleges
5 features
courses departments enrollments redesign_audit_events redesign_outbox_events

Analytics alert rules

€ R U D

2 endpoints · /admin/analytics

R GET /api/admin/alert-rules

list alert thresholds

U PUT /api/admin/alert-rules

set alert threshold

SCREENS /admin/analytics

CODE routers/console.py

GATE require_capability(admin.analytics) · capability admin.analytics

TABLES alert_rule_configs cohorts

AWS none

Charts & numbers (analytics)

€ R U D

5 endpoints · /admin/analytics

R GET /api/admin/analytics-summary

read analytics header counts

R GET /api/admin/analytics/kpis

read KPI strip

R GET /api/admin/analytics/series

read weekly trend lines

R GET /api/admin/students/{student_id}/resume.pdf

download student CV PDF

R GET /api/admin/students/{student_id}/weekly

read student six-week trend

SCREENS /admin/analytics /admin/students/:id

CODE routers/console.py

GATE require_capability(admin.analytics) + _assert_can_access_student ·
require_capability(admin.analytics) + scope_filter(admin.analytics) · capability
admin.analytics

TABLES attendance_records badge_evidence certification_progress placement_criteria
placement_offers registrations resumes semester_results skills student_badges
student_skills students time_ledger_cells time_ledger_days time_sheet_entries users

AWS PDF render (in process)

Download reports



6 endpoints · /admin/exports

- R** **GET** `/api/admin/badges/cohort`
read cohort badge summary **API ONLY**
- R** **GET** `/api/admin/badges/export.csv`
download cohort skill CSV
- R** **GET** `/api/admin/exports/history`
list export receipts
- R** **GET** `/api/admin/exports/ledger.csv`
download ledger compliance CSV
- R** **GET** `/api/admin/exports/placement.csv`
download placement CSV
- R** **GET** `/api/admin/exports/students.csv`
download student-mentor map CSV

SCREENS `/admin/exports` `/admin/placement`

CODE `routers/badge_verification.py` `routers/console.py`

GATE `require_admin` · `require_admin + scope_filter(admin.exports)` ·
`require_capability(admin.exports) + scope_filter(admin.exports)` · `capability` `admin.exports`

TABLES `capability_assessments` `cohorts` `export_events` `mentors` `placement_offers`
`redesign_audit_events` `redesign_outbox_events` `student_badges` `students` `time_ledger_cells`
`time_ledger_days` `users`

AWS none

Student alerts



2 endpoints · /admin/analytics

- R** **GET** `/api/mentor/alerts`
list student alerts
- U** **POST** `/api/mentor/alerts/{alert_id}/resolve`
resolve alert **API ONLY**

SCREENS `/admin/analytics`

CODE `routers/mentor.py`

GATE `require_mentor` · `require_mentor + _assert_can_access_student`

TABLES `alerts` `students` `users`

AWS none

Upload spreadsheets (imports)

C R U D

5 endpoints · /admin/imports

- C** **POST** /api/admin/imports/preview
check import file
- C** **POST** /api/admin/imports/{run_id}/apply
import rows into records
- R** **GET** /api/admin/imports
list recent imports
- R** **GET** /api/admin/imports/templates/{kind}.xlsx
download import template
- R** **GET** /api/admin/imports/{run_id}/errors.csv
download error report

SCREENS /admin/imports

CODE routers/admin_imports.py

GATE require_capability(admin.imports) · require_capability(admin.imports) +
require_capability(admin.imports, target=ancestry_of_cohort) ·
require_capability(admin.imports) + require_capability(admin.imports,
target=ancestry_of_cohort) (student names only with admin.students) ·
require_capability(admin.imports) + scope_filter · capability admin.imports

TABLES attendance_records capability_grants cohorts courses export_events import_rows
import_runs redesign_audit_events redesign_outbox_events semester_results students
subject_marks users

AWS none

Governance & audit 4 features

College admins

C R U D

2 endpoints · /admin/colleges

- C** **POST** /api/admin/colleges/{college_id}/admins
appoint college admin
- R** **GET** /api/admin/colleges/{college_id}/admins
list college admins

SCREENS /admin/colleges

CODE routers/admin.py

GATE require_governance (require_staff + require_capability(admin.governance)) · capability
admin.governance

TABLES capability_grants colleges redesign_audit_events redesign_outbox_events users

AWS none

Student feature switches

€ R U D

3 endpoints · /admin/governance

- R** **GET** /api/admin/governance/features
list feature switches
- U** **PUT** /api/admin/governance/features
set feature switch
- D** **DELETE** /api/admin/governance/features/{override_id}
clear feature switch

SCREENS /admin/governance /admin/governance/features

CODE routers/governance.py

GATE require_governance = policies.require_staff + require_capability(admin.governance) · capability
admin.governance

TABLES academic_courses academic_specializations cohorts colleges departments
feature_overrides redesign_audit_events redesign_outbox_events students users

AWS none

What changed (audit log)

€ R U D

3 endpoints · /admin/audit

- R** **GET** /api/admin/audit
list audit events
- R** **GET** /api/admin/audit/export.csv
export audit log CSV
- R** **GET** /api/admin/audit/{event_id}
open audit event

SCREENS /admin/audit

CODE routers/audit.py

GATE _gate → require_admin

TABLES redesign_audit_events redesign_outbox_events users

AWS none

Who can do what (Governance)

C R U D

14 endpoints · /admin/governance

C	POST	/api/admin/governance/grants	grant capability
C	POST	/api/admin/governance/groups	create access group
C	POST	/api/admin/governance/groups/{group_id}/members	add group members
R	GET	/api/admin/governance/catalogue	list capabilities and features
R	GET	/api/admin/governance/effective/{user_id}	show effective capabilities API ONLY
R	GET	/api/admin/governance/grants	list live grants
R	GET	/api/admin/governance/groups	list access groups
R	GET	/api/admin/governance/hierarchy	list scope targets
R	GET	/api/admin/governance/review	list grants to review
R	GET	/api/admin/governance/staff	list grantable staff
U	POST	/api/admin/governance/grants/{grant_id}/approve	approve pending grant
U	POST	/api/admin/governance/grants/{grant_id}/extend	extend grant
D	POST	/api/admin/governance/grants/{grant_id}/revoke	revoke grant
D	DELETE	/api/admin/governance/groups/{group_id}/members/{user_id}	remove group member

SCREENS `/admin/governance` `/admin/governance/features` `/admin`

CODE `routers/governance.py`

GATE `require_governance = policies.require_staff + require_capability(admin.governance) · capability`
`admin.governance`

TABLES `academic_courses` `academic_specializations` `access_group_members` `access_groups`
`capability_grants` `cohorts` `colleges` `departments` `redesign_audit_events`
`redesign_outbox_events` `students` `users`

AWS `none`

Deletion & retention 3 features

Delete a college

2 endpoints · /admin/colleges

R

GET

/api/admin/colleges/{college_id}/delete-plan

preview college deletion

D

POST

/api/admin/colleges/{college_id}/delete

delete college permanently

SCREENS

/admin/colleges

CODE

routers/admin_deletion.py

GATE

require_admin · require_admin + emailed deletion code

TABLES

academic_calendar academic_courses academic_specializations alert_rule_configs auth_tokens badge_course_map cohorts colleges departments interview_policies interview_tracks jobs redesign_audit_events redesign_outbox_events registrations stage_rules students users

AWS

none

Deletion code

1 endpoint · /admin/students

A

POST

/api/admin/deletions/code

email deletion code

SCREENS

/admin/students /admin/faculty /admin/colleges

CODE

routers/admin_deletion.py

GATE

require_admin (throttled 6/hour per admin)

TABLES

auth_tokens mail_logs users

AWS

Amazon SES mail

Remove / delete people

€ R U D

8 endpoints · /admin/students

- R** **GET** /api/admin/students/{student_id}/delete-plan
preview student deletion
- R** **GET** /api/admin/users/{user_id}/delete-plan
preview account deletion
- U** **POST** /api/admin/students/{student_id}/restore
restore removed student
- U** **POST** /api/admin/users/{user_id}/restore
restore removed account
- D** **POST** /api/admin/students/{student_id}/delete
delete student permanently
- D** **POST** /api/admin/students/{student_id}/remove
remove student (soft)
- D** **POST** /api/admin/users/{user_id}/delete
delete account permanently
- D** **POST** /api/admin/users/{user_id}/remove
remove account (soft)

SCREENS /admin/students /admin/faculty

CODE routers/admin_deletion.py

GATE require_admin · require_admin + emailed deletion code

TABLES archived_documents attendance_records auth_tokens badge_evidence capability_grants
conversations interview_sessions leave_requests mail_logs mentor_assignments
mentor_notes mentors platform_call_sessions redesign_audit_events
redesign_mentor_notebook_entries redesign_outbox_events registrations resumes
semester_results staff_signatures staff_upskilling_certs student_badges students
swoc_entries uploads users

AWS EFS /data volume · S3 call recordings

Voice platform call sessions

4 endpoints · API only

€

R

U

Ð

R

GET

/api/platform/admin/calls

list recent calls

API ONLY

R

GET

/api/platform/calls

list call sessions

API ONLY

R

GET

/api/platform/calls/{session_id}

view call detail

API ONLY

U

POST

/api/platform/calls/{session_id}/close

close call, upload recording

API ONLY

SCREENS

none for this role (API only)

CODE

voice_platform/api/admin.py

voice_platform/api/calls.py

GATE

get_current_session (any signed-in session); role = ADMIN lists every call, every other role is filtered to user_id = session.userId · get_current_session + _can_read (owner | role ADMIN | MENTOR via _assert_can_access_student on the linked interview's student); presigned recording URL only if has_capability(admin.interview_audio) · get_current_session + owner (row.user_id = session.userId) or role ADMIN, else 403; 409 if running on another worker · require_admin (via the router's _admin dependency) · capability admin.interview_audio

TABLES

capability_grants

interview_sessions

interview_turns

mentors

platform_call_sessions

platform_recording_policies

students

AWS

CloudWatch metrics · DynamoDB voice sessions · EFS /data volume · S3 call recordings · Sentry

Voice platform candidates

C R U D

6 endpoints · API only

- C** **POST** `/api/platform/admin/candidates`
add candidate **API ONLY**
- C** **POST** `/api/platform/admin/candidates/bulk`
bulk upload candidates **API ONLY**
- R** **GET** `/api/platform/admin/candidates`
list candidates **API ONLY**
- U** **PATCH** `/api/platform/admin/candidates/{candidate_id}`
edit candidate **API ONLY**
- U** **POST** `/api/platform/admin/candidates/{candidate_id}/link`
link candidate login **API ONLY**
- D** **DELETE** `/api/platform/admin/candidates/{candidate_id}`
delete candidate **API ONLY**

SCREENS none for this role (API only)

CODE `voice_platform/api/admin.py`

GATE `require_admin` (via the router's `_admin` dependency)

TABLES `platform_call_sessions` `platform_candidates` `platform_specializations` `users`

AWS CloudWatch metrics · SQS candidate queues

Voice platform question bank

C R U D

4 endpoints · API only

- C** **POST** `/api/platform/admin/specializations/{spec_id}/questions`
add question **API ONLY**
- R** **GET** `/api/platform/admin/specializations/{spec_id}/questions`
list questions **API ONLY**
- U** **PATCH** `/api/platform/admin/questions/{question_id}`
edit question **API ONLY**
- D** **DELETE** `/api/platform/admin/questions/{question_id}`
delete question **API ONLY**

SCREENS none for this role (API only)

CODE `voice_platform/api/admin.py`

GATE `require_admin` (via the router's `_admin` dependency)

TABLES `platform_questions` `platform_specializations`

AWS CloudWatch metrics

Voice platform recording policies

€ R U ⌕

2 endpoints · API only

- R** **GET** `/api/platform/admin/recording-policies`
list recording policies **API ONLY**
- U** **PUT** `/api/platform/admin/recording-policies/{degree}`
set recording policy **API ONLY**

SCREENS none for this role (API only)

CODE `voice_platform/api/admin.py`

GATE `require_admin` (via the router's `_admin` dependency)

TABLES `platform_recording_policies`

AWS CloudWatch metrics

Voice platform specializations

C R U D

5 endpoints · API only

- C** **POST** `/api/platform/admin/specializations`
create specialization **API ONLY**
- R** **GET** `/api/platform/admin/specializations`
list specializations **API ONLY**
- R** **GET** `/api/platform/admin/specializations/{spec_id}`
view specialization **API ONLY**
- U** **PATCH** `/api/platform/admin/specializations/{spec_id}`
edit specialization **API ONLY**
- D** **DELETE** `/api/platform/admin/specializations/{spec_id}`
delete specialization **API ONLY**

SCREENS none for this role (API only)

CODE `voice_platform/api/admin.py`

GATE `require_admin` (via the router's `_admin` dependency)

TABLES `platform_questions` `platform_specializations` `platform_time_limits`

AWS CloudWatch metrics

Voice platform status

€ R U D

1 endpoint · API only

R GET /api/platform/admin/status

check platform configuration API ONLY

SCREENS none for this role (API only)

CODE voice_platform/api/admin.py

GATE require_admin (via the router's _admin dependency)

TABLES none

AWS none

Voice platform time limits

€ R U D

3 endpoints · API only

R GET /api/platform/admin/time-limits

list time limits API ONLY

U PUT /api/platform/admin/time-limits

set time limit API ONLY

D DELETE /api/platform/admin/time-limits/{limit_id}

delete time limit API ONLY

SCREENS none for this role (API only)

CODE voice_platform/api/admin.py

GATE require_admin (via the router's _admin dependency)

TABLES platform_specializations platform_time_limits

AWS none

EVERY SIGNED-IN ACCOUNT

Things every signed-in account does to itself: session, sign-out, password, linked Google, notification preferences.

GATE `get_current_session` only — any role. The account itself: session, sign-out, password, linked Google, notification preferences.

5 features 13 endpoints features that 1 read 2 update 1 delete 3 action

Identity & sign-in 4 features

Change password

€ R **U** D A

3 endpoints · `/account/password`

- U** **POST** `/api/auth/change-password`
change my password
- A** **POST** `/api/auth/change-password/code`
email change-password code
- A** **POST** `/api/auth/forgot`
email me a setup link (no password yet)

SCREENS `/account/password` `/account`

CODE `routers/passwords.py`

GATE `get_current_session` · `get_current_session + exactly one proof (emailed code or current password)` · `none` (rate-limited per address and globally)

TABLES `auth_tokens` `mail_logs` `users`

AWS Amazon SES mail

Current session

€ **R** U D

2 endpoints · `/account`

- R** **GET** `/api/auth/me`
read my session
- R** **GET** `/api/v1/auth/me`
read my session **API ONLY**

SCREENS `/account` `/account/password` (app shell, every signed-in screen)

CODE `routers/auth.py`

GATE `get_current_session`

TABLES `access_group_members` `capability_grants` `feature_overrides` `login_events` `mentors` `students` `users`

AWS none

Linked Google & sessions

€ R U **D** A

4 endpoints · /account

- D** **POST** /api/auth/google/unlink
unlink Google sign-in
- D** **POST** /api/v1/auth/google/unlink
unlink Google sign-in **API ONLY**
- A** **POST** /api/auth/sign-out-everywhere
sign out everywhere
- A** **POST** /api/v1/auth/sign-out-everywhere
sign out everywhere **API ONLY**

SCREENS /account

CODE routers/auth.py

GATE get_current_session · get_current_session (refused 409 unless the account holds a real password and the password door is open)

TABLES redesign_audit_events redesign_outbox_events users

AWS none

Sign out

€ R U **D** **A**

2 endpoints · /account

- A** **POST** /api/auth/logout
sign out
- A** **POST** /api/v1/auth/logout
sign out **API ONLY**

SCREENS /account (app shell, every signed-in screen)

CODE routers/auth.py

GATE none (cookie read optionally; idempotent)

TABLES users

AWS none

Notification preferences

€

R

U

Ð

2 endpoints · /account

U

PUT

/api/auth/notification-prefs

set notification preferences

U

PUT

/api/v1/auth/notification-prefs

set notification preferences

API ONLY

SCREENS

/account

CODE

routers/auth.py

GATE

get_current_session (unknown or unenforced keys refused 422)

TABLES

redesign_audit_events

redesign_outbox_events

users

AWS

none

PUBLIC – no session

No session at all: applicants, the sign-in doors, onboarding, password reset, health probes.

GATE No session cookie. Applicants, the sign-in doors, onboarding, password reset and the load balancer's health probe.

9 features 29 endpoints features that 1 create 3 read 4 update 5 action

Identity & sign-in 6 features

Google sign-in

€ R U D A

8 endpoints · /login

A

GET

/api/auth/google/callback

finish Google sign-in

API ONLY

A

GET

/api/auth/google/start

start Google sign-in

API ONLY

A

GET

/api/auth/sso/google

start Google sign-in

A

GET

/api/auth/sso/google/callback

finish Google sign-in

API ONLY

A

GET

/api/v1/auth/google/callback

finish Google sign-in

API ONLY

A

GET

/api/v1/auth/google/start

start Google sign-in

API ONLY

A

GET

/api/v1/auth/sso/google

start Google sign-in

API ONLY

A

GET

/api/v1/auth/sso/google/callback

finish Google sign-in

API ONLY

SCREENS

/login

CODE

routers/auth.py

GATE

none (public; refuses with /login?error=sso_config when Google is not configured) · none (public; single-use state cookie + nonce, verified Google ID token, roster lookup, google_sub pin)

TABLES

login_days login_events mail_logs mentors students users

AWS

Google OAuth / OIDC · Amazon SES mail

Password reset

€ R **U** D A

2 endpoints · /reset

U **POST** /api/auth/reset

reset password from link

A **POST** /api/auth/forgot

request password reset link

SCREENS /reset /login

CODE routers/passwords.py

GATE none (rate-limited per address and globally) · reset token (auth_tokens, PURPOSE_RESET);
refuses a barred account

TABLES auth_tokens mail_logs users

AWS Amazon SES mail

Password sign-in

€ R U D **A**

4 endpoints · /login

A **POST** /api/auth/login

sign in with password

A **POST** /api/auth/login/code

verify emailed login code

A **POST** /api/v1/auth/login

sign in with password API ONLY

A **POST** /api/v1/auth/login/code

verify emailed login code API ONLY

SCREENS /login

CODE routers/auth.py

GATE none (public; password_door_open(db), per-account brute-force limiter,
refuse_disabled_sign_in) · none (public; per-account code limiter, consume_user_code,
refuse_disabled_sign_in)

TABLES auth_tokens capability_grants login_days login_events mail_logs mentors students
users

AWS Amazon SES mail

Sign out

€ R U D A

2 endpoints · API only

A **POST** /api/auth/logout

sign out API ONLY

A **POST** /api/v1/auth/logout

sign out API ONLY

SCREENS none for this role (API only)

CODE routers/auth.py

GATE none (cookie read optionally; idempotent)

TABLES users

AWS none

Sign-in options

€ R U D

4 endpoints · /login

R **GET** /api/auth/google/status

check sign-in options API ONLY

R **GET** /api/auth/sso/status

check sign-in options

R **GET** /api/v1/auth/google/status

check sign-in options API ONLY

R **GET** /api/v1/auth/sso/status

check sign-in options API ONLY

SCREENS /login

CODE routers/auth.py

GATE none (public, unauthenticated by design)

TABLES users

AWS none

Staff account activation Registration & onboarding

2 features

€ R U D

1 endpoint · /activate

Registration form

C R U D

3 endpoints · /register

- C **POST** /api/register
submit application
- R **GET** /api/register/hierarchy
list college hierarchy
- U **POST** /api/register/{registration_id}/documents/{kind}
replace application document **API ONLY**

SCREENS /register

CODE routers/registration.py

GATE none (public) · none (public; application id as bearer, per-IP rate limit, undecided applications only) · none (public; per-IP rate limit, duplicate guard)

TABLES academic_courses academic_specializations archived_documents auth_tokens cohorts colleges departments mail_logs registration_documents registration_rules registrations student_profiles students uploads users

AWS EFS /data volume · S3 documents archive · Amazon SES mail

Student account setup (onboarding)

€ R U D A

3 endpoints · /onboard

- U **POST** /api/auth/onboard/password
set first password (after code)
- A **POST** /api/auth/onboard/start
confirm address, send code
- A **POST** /api/auth/onboard/verify
verify emailed code

SCREENS /onboard

CODE routers/onboarding.py

GATE none (proof is the PURPOSE_ONBOARD setup-link token + matching email typed back; barred accounts refused) · none (proof is the PURPOSE_ONBOARD_SET ticket from step 2; barred accounts refused; password_problem policy) · none (proof is the setup-link token + the emailed PURPOSE_ONBOARD_CODE, consumed atomically)

TABLES auth_tokens mail_logs users

AWS Amazon SES mail

Health probes

2 endpoints · API only

R

GET

/health

liveness probe

API ONLY

R

GET

/ready

readiness probe

API ONLY

SCREENS

none for this role (API only)

CODE

router/health.py

GATE

none (unauthenticated)

TABLES

none

AWS

none

Stack & AWS

The path a request takes, and everything around the API task: data, files, mail, AI, observability, schedules, backups and the deploy pipeline.

One request, phone to role

Same origin under `/api`, so the `httpOnly reep_session` cookie rides every call. CloudFront's default behaviour serves the SPA from S3; `/api/*` (HTTP and the interview WebSockets) goes to the ALB and an ECS Fargate task running `uvicorn + FastAPI`. `security.py` verifies the HS256 JWT and the account's `token_version` (one live session per account), then the role decides the lane.

- **A person opens REEP**
phone browser or installed PWA
- **Angular 22 SPA (apps/web)**
`fetch('/api/...', credentials: include)`
- **Service worker reep-sw.js**
`/api/*` passes through untouched
- **DNS reep.sast-skills.com**
external zone → CloudFront
- **AWS WAF web ACL reep-edge**
us-east-1: common, bad-inputs, rate 2000/5 min
- **CloudFront EW482RJK9CVE**
TLS 1.3, alias reep.sast-skills.com
- **S3 SPA bucket reep-web**
static files: OAC, hashed assets immutable 1 y
- **ALB reep-alb :443**
CloudFront prefix list only; TG reep-api :3300
- **ECS Fargate · service api**
ARM64, 512 CPU / 1 GiB, 2-10 tasks
- **uvicorn → FastAPI app.main**
`python -m uvicorn app.main:app :3300`
- **Middleware chain**
Sentry wrap, headers, X-Request-ID, CORS
- **security.py · verify reep_session**
HS256 AUTH_SECRET + users.token_version
- **Who is calling? (users.role in the session)**
cookie absent → PUBLIC; 401 X-Reep-Session: retired

Then the role decides the lane: **Student** **Faculty** **Alumni** **Main Admin** **Any**
account **Public**

What the API task talks to

Every call to AWS APIs or the internet leaves the private subnets through the NAT instance (t4g.nano) — there are no VPC endpoints; RDS and EFS traffic stays inside the VPC. Stacks: `reep-core` (ap-south-1), `reep-edge-waf` (us-east-1), `reep-dr-vault` (ap-southeast-1), `reep-voice-platform` (ap-south-1). Every resource in `reep-core`, `reep-edge-waf` and `reep-dr-vault` is DeletionPolicy Retain; `reep-voice-platform` retains only its two S3 buckets and two DynamoDB tables.

ECR `reep/api` (multi-arch)

image pulled at task start

→ ECS task `reep-api` (api service + jobs)

Edge request logs

CloudFront + ALB access logs

→ S3 `reep-cloudfront-logs`, S3 `reep-alb-logs`

S3 `reep-cloudfront-logs`

no query/cookie/referer; 30 d (us-east-1)

Secrets Manager `reep/app` + `reep/external`

AUTH_SECRET, DATABASE_URL, Google, Sentry

→ ECS task `reep-api` (api service + jobs)

S3 `reep-alb-logs`

expire 90 d

ECS task `reep-api` (api service + jobs)

task role `reep-api-task`, no keys

→ RDS PostgreSQL 17 `reep-postgres`, EFS `reep-data` → `/data`, S3 `reep-documents-archive`, Amazon SES `no-reply@sast-skills.com`, Bedrock Nova 2 Sonic (ap-northeast-1), Bedrock Nova Pro (Converse), Google OAuth / OIDC, Sentry `reep-api` / `reep-web`, CloudWatch Logs `/reep/api`

RDS PostgreSQL 17 `reep-postgres`

db `reep_py`, :5432 from api SG only

EFS `reep-data` → `/data`

uploads + interview-audio, IA after 30 d

S3 `reep-documents-archive`

Object Lock 10 y, Put + List only

Amazon SES `no-reply@sast-skills.com`

config set `reep-transactional`

→ SNS `reep-ses-notifications`

SNS reep-ses-notifications

SES event stream + rate alarms; hand-made, no stack

Bedrock Nova 2 Sonic (ap-northeast-1)

mock interviewer, 8-min stream cap

Bedrock Nova Pro (Converse)

REEP Agent, resume brief

Google OAuth / OIDC

sign-in for every role

Sentry reep-api / reep-web

PII off, scrubbed payloads

CloudWatch Logs /reep/api

metric filters: dropped turns, mail failed

→ CloudWatch alarms

CloudWatch alarms

5xx, no healthy api, RDS, backups

→ SNS reep-alerts (email)

SNS reep-alerts (email)

ops mailbox

Route 53 health check reep-uptime

TCP 443 every 30 s (us-east-1)

→ Alarm reep-site-unreachable

Alarm reep-site-unreachable

missing data = BREACHING (us-east-1)

→ SNS reep-uptime-alerts

SNS reep-uptime-alerts

email, confirm subscription (us-east-1)

What runs at night

Same image, roles and EFS mount as the api. Order is load-bearing: the ledger (23:30) and the dump (01:00) and the archive sweep (02:00) all run BEFORE retention (03:00), the one scheduled destructor — a copy taken after it never saw what it removed.

Writers may PutObject and List, never Get or Delete.

23:30 IST · reep-identity-ledger-daily

EventBridge cron(0 18 * * ? *) UTC

→ RunTask · python -m app.export_identity

RunTask · python -m app.export_identity

READ: one JSON per account, refuses an empty export

→ Identity ledger → PostgreSQL READ, Identity ledger → S3 reep-identity-ledger

Identity ledger → PostgreSQL READ

users, students, institution labels

Identity ledger → S3 reep-identity-ledger

Object Lock 10 y, PutObject only

01:00 IST · reep-db-dump-daily

EventBridge cron(30 19 * * ? *) UTC

→ RunTask · python -m app.backup_database

RunTask · python -m app.backup_database

READ: pg_dump -Fc, verified with pg_restore --list

→ Logical dump → PostgreSQL pg_dump, Logical dump → S3 reep-db-dumps, Logical dump → S3 reep-db-archive

Logical dump → PostgreSQL pg_dump

whole database

Logical dump → S3 reep-db-dumps

lock 90 d, expire 97 d

Logical dump → S3 reep-db-archive

lock 10 y; Put + List (monthly/), never Get

02:00 IST · reep-document-archive-daily

EventBridge cron(30 20 * * ? *) UTC

→ RunTask · python -m app.archive_documents

RunTask · python -m app.archive_documents

CREATE: uploads what the archive lacks; never deletes

→ Document archive → EFS /data, Document archive → S3 reep-documents-archive

Document archive → EFS /data

document stores + interview WAVs

Document archive → S3 reep-documents-archive

Put + List, never Get

03:00 IST · reep-retention-daily

EventBridge cron(30 21 * * ? *) UTC

→ RunTask · python -m app.retention_job

RunTask · python -m app.retention_job

DELETE: the one scheduled destructor (180-day interviews)

→ Retention → PostgreSQL DELETE, Retention → EFS delete files

Retention → PostgreSQL DELETE

auth_tokens, registration_documents, interview_*

Retention → EFS delete files

interview audio + rejected applicants' files

07:00 IST · reep-leave-today-daily

EventBridge cron(30 1 * * ? *) UTC

→ RunTask · python -m app.leave_today_job

RunTask · python -m app.leave_today_job

CREATE: '<name> is on leave today' to faculty

→ On leave today → PostgreSQL, On leave today → Amazon SES

On leave today → PostgreSQL

READ leave_requests, users; WRITE mail_logs

On leave today → Amazon SES

suppression checked first

Alert engine (NOT scheduled) NOT RUNNING

intended 02:00 IST; no schedule in the stack

→ RunTask · python -m app.alerts_job

RunTask · python -m app.alerts_job

CREATE: CLI only today

→ Alert engine → PostgreSQL alerts

Alert engine → PostgreSQL alerts

alerts, alert_rule_configs

Backups and disaster recovery

PHYSICAL copies (AWS Backup, 35 days = RDS's ceiling, plus a monthly 365-day archive of RDS only) and the LOGICAL ones above (identity ledger, pg_dump daily + monthly archive, documents archive) — the logical half is what survives an engine or vendor change.

AWS Backup daily 00:30 IST

plan reep-daily: RDS + EFS, 35 d

→ Backup source: RDS reep-postgres, Backup source: EFS reep-data

AWS Backup monthly 21:30 IST on the 1st

plan reep-archive: RDS only, 365 d

→ Backup source: RDS reep-postgres

Backup source: RDS reep-postgres

daily + monthly selections

→ Backup vault reep-vault

Backup source: EFS reep-data

daily selection only

→ Backup vault reep-vault

Backup vault reep-vault

vault lock governance, min 35 d

→ DR vault reep-vault-dr (Singapore)

DR vault reep-vault-dr (Singapore)

cross-region copy

Restore test Sundays 09:30 IST

throwaway RDS from latest point

→ Restore from the newest point in reep-vault

Restore from the newest point in reep-vault

throwaway instance, 1 h validation

Voice platform

Bulk candidate files land in S3; the Lambda validates each row and queues it per degree. **No queue consumer is deployed** — the admin bulk upload stores straight into Postgres by default. Calls run through `/api/platform/media-bridge` inside the api task (lane MAIN ADMIN and STUDENT), recordings go to S3, live state is projected to DynamoDB.

S3 uploads/ (ObjectCreated)

CSV / JSON / JSONL

→ Lambda reep-voice-candidate-ingest

Lambda reep-voice-candidate-ingest

validation.py, rejects/ report

→ SQS reep-voice-candidates-ug / -pg

api task: media bridge + bulk upload

/api/platform/* inside reep-api (section 2)

→ SSM /reep/voice-platform/PLATFORM_*, DynamoDB reep-voice-sessions-ug / -pg, S3 recordings/ (versioned), SQS reep-voice-candidates-ug / -pg

SQS reep-voice-candidates-ug / -pg

visibility 60 s, 4 d

→ SQS dead-letter queues, Queue worker (NOT deployed)

SQS dead-letter queues

after 5 receives, 14 d

Queue worker (NOT deployed) NOT RUNNING

no service or schedule runs it

SSM /reep/voice-platform/PLATFORM_*

read by the api at boot

DynamoDB reep-voice-sessions-ug / -pg

TTL 180 d, PITR

S3 recordings/ (versioned)

no lifecycle; presigned GET

Shipping it

ci.yml runs the five checks the (not yet applied) ruleset in

.github/rulesets/main.json will require. deploy.yml assumes reep-github-deploy by OIDC and runs two jobs in PARALLEL: api (push the multi-arch image, alembic upgrade head as a one-off task BEFORE rolling the service) and web (publish the SPA, invalidate CloudFront). cdk-deploy.yml and ops-task.yml are human-dispatched only; agent-release.yml dispatches deploy only when vars.AGENT_AUTODEPLOY is true AND tools/ci/release_gate.py allows it.

Push / PR on darshani8/reep-

main NOT protected yet: ruleset not applied

→ ci.yml · api · pii-gate · api-imports · web · cdk

ci.yml · api · pii-gate · api-imports · web · cdk

pytest, PII gate, imports, ng build, synth

→ agent-release.yml (release gate)

agent-release.yml (release gate)

migrations / infra / auth always need a human

→ deploy.yml

deploy.yml

workflow_dispatch: api / web / both

→ IAM OIDC → role reep-github-deploy

cdk-deploy.yml (human only)

voice / edge-waf / dr-vault / core-9a / core-9b / NAT x2

→ IAM OIDC → role reep-github-deploy

ops-task.yml (fixed task menu)

purge-*, seed-*, grant-access as RunTask

→ IAM OIDC → role reep-github-deploy

infra-drift.yml 08:00 IST

cdk diff vs main + drift + cost; never deploys

→ IAM OIDC → role reep-github-deploy

IAM OIDC → role reep-github-deploy

ECR, ECS, PassRole, S3 SPA, CF, CE; CDK boot roles

→ ECR push reep/api:<sha> + :latest, S3 sync SPA reep-web

ECR push reep/api:<sha> + :latest

multi-arch; amd64 + arm64 asserted

→ RunTask alembic upgrade head

RunTask alembic upgrade head

fails before the roll

→ ECS update-service api

ECS update-service api

circuit breaker + rollback

S3 sync SPA reep-web

index / ngsw / reep-sw no-cache

→ CloudFront invalidation

claude.yml / claude-review.yml

@claude → PR; review every non-draft PR

agent-ops.yml → agent-triage.yml

drift, failed deploys, incidents

agent-maintenance.yml Mondays

dependency and drift sweep

CloudFront invalidation

/, /index.html, /ngsw.json, /ngsw-worker.js, /reep-sw.js

Operator task menu

`ops-task.yml` is a fixed list; each entry runs `python -m app.<module>` as a one-off task on the API image, and a purge also asks for its own typed sentence.

`seed-roster-dry-run` `seed-roster` `seed-kb` `seed-catalogue-dry-run` `seed-catalogue` `grant-access`
`purge-people-dry-run` `purge-people` `purge-students-dry-run` `purge-students` `purge-colleges-dry-run`
`purge-colleges`

Every table in RDS 112 tables

Academics & attendance

8 tables

`attendance_records` `courses` `enrollments` `lab_sessions` `schedule_items` `semester_results`
`student_semester_history` `subject_marks`

Documents & uploads

2 tables

`archived_documents` `uploads`

Governance & audit

8 tables

`access_group_members` `access_groups` `capability_grants` `export_events` `feature_overrides`
`redesign_audit_events` `redesign_tenant_memberships` `redesign_tenants`

Identity & auth

4 tables

`auth_tokens` `login_days` `login_events` `users`

Imports & analytics

5 tables

`alert_rule_configs` `alerts` `analytics_snapshots` `import_rows` `import_runs`

Institution spine

6 tables

`academic_courses` `academic_specializations` `cohorts` `colleges` `departments` `stage_rules`

Interviews

9 tables

interview_bank_questions interview_cap_resets interview_consents interview_evaluations
interview_policies interview_score_summaries interview_sessions interview_tracks interview_turns

Jobs & placement

5 tables

job_applications jobs mock_attempts placement_criteria placement_offers

Knowledge base & agent

11 tables

agent_runs assistant_feedback conversations knowledge_chunks knowledge_documents messages
redesign_embedding_models redesign_knowledge_chunk_embeddings redesign_knowledge_chunks
redesign_knowledge_document_versions redesign_knowledge_namespaces

Leave

5 tables

academic_calendar leave_attachments leave_balances leave_requests staff_signatures

Mail & notifications

1 table

mail_logs

Mentoring & SWOC

10 tables

mentor_assignments mentor_notes mentors redesign_mentor_notebook_actions
redesign_mentor_notebook_attachments redesign_mentor_notebook_entries
redesign_mentor_notebook_entry_revisions staff_upskilling_certs swoc_entries swoc_entry_revisions

Other

5 tables

alembic_version redesign_api_idempotency_keys redesign_domain_jobs redesign_outbox_events
students_orphaned_cohort_ids

Programme (ledger/english/milestones)

6 tables

english_baseline_sections english_baselines student_milestones time_ledger_cells time_ledger_days
time_sheet_entries

Registration

4 tables

email_verifications registration_documents registration_rules registrations

Skills/badges/certifications

10 tables

approved_certifications badge_course_map badge_evidence capability_assessments certification_progress
certifications skill_claims skills student_badges student_skills

Students & profile

7 tables

academic_gaps academic_qualifications alumni_profiles resume_profiles resumes student_profiles
students

Voice platform

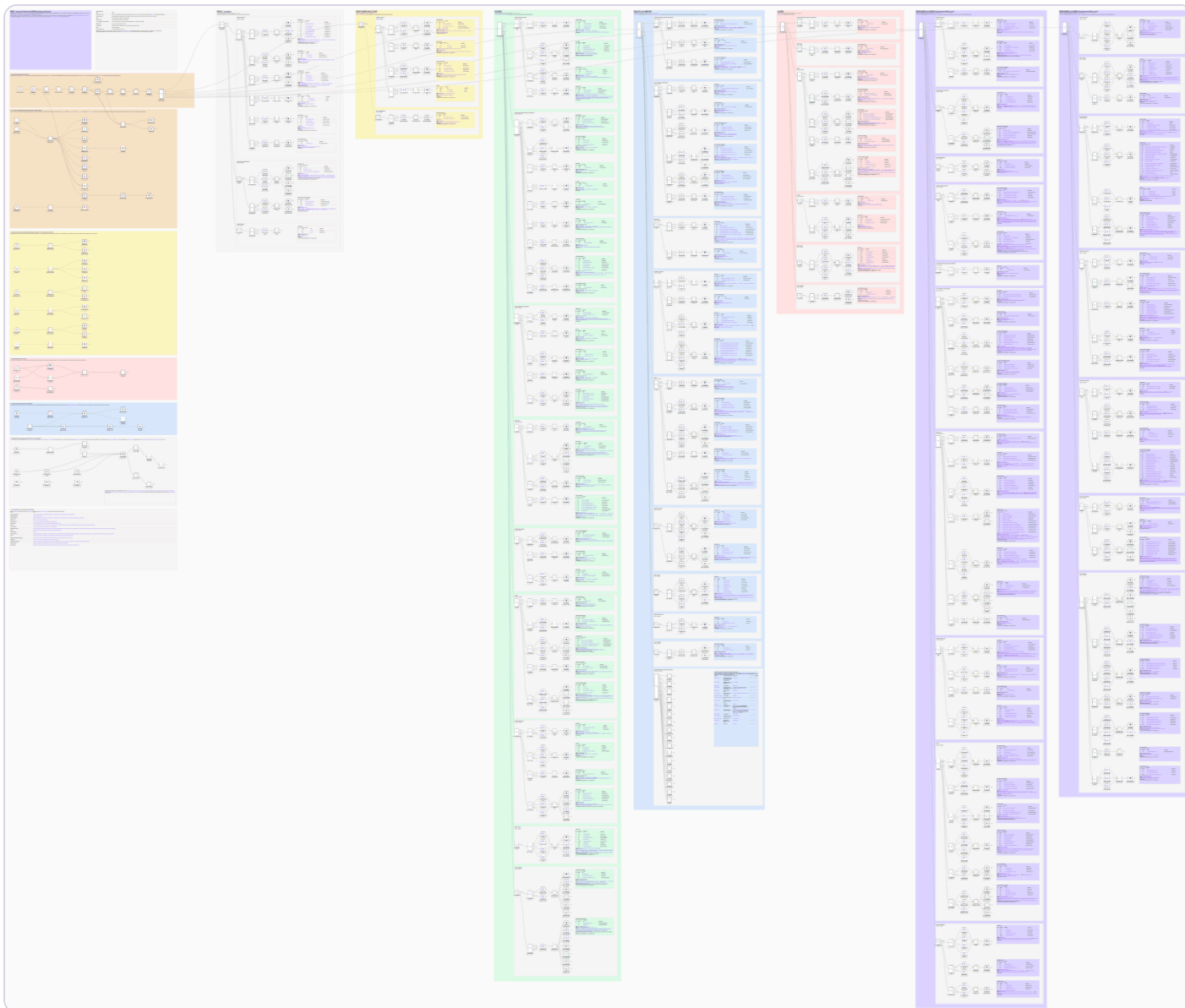
6 tables

platform_call_sessions platform_candidates platform_questions platform_recording_policies
platform_specializations platform_time_limits

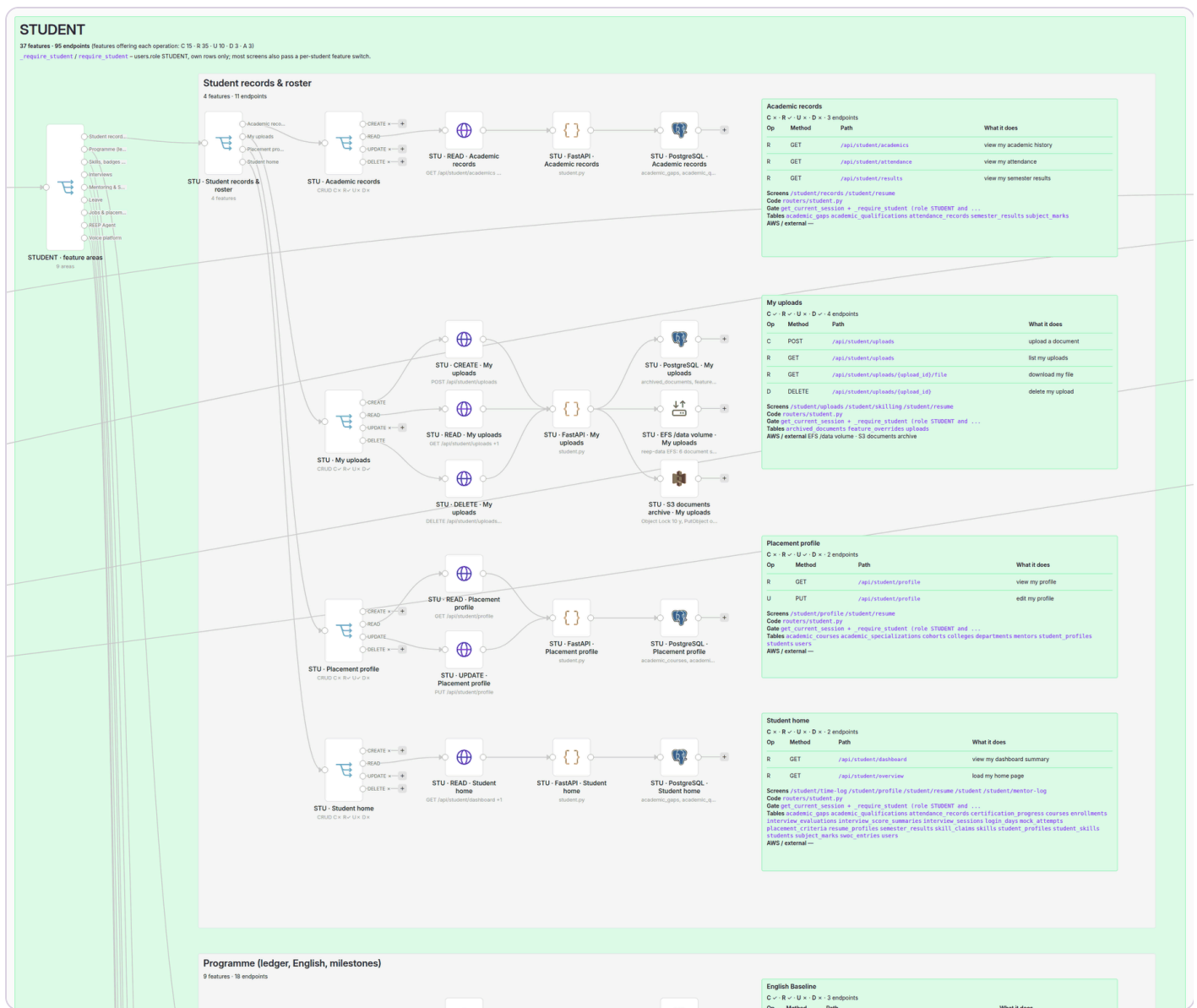
The n8n diagram

The same map as rendered by n8n 2.41.6 from reep-roles-features-crud.n8n.json:
the whole canvas, the request path and AWS estate, and the top of the Student lane.

Import the .json into n8n (Workflows, Import from File) to explore it.



The whole canvas, zoomed to fit: the request path and AWS estate on the left, one lane per role to the right.



The top of the Student lane: role, feature area, feature switch with its CREATE / READ / UPDATE / DELETE outputs, API call, FastAPI handler, tables and AWS services, and the feature card.

C **R** **U** **D** A filled letter is an operation this role is offered; a struck-out one is not. **A** marks an action such as a sign-in, a code or a live interview. **API ONLY** marks an endpoint no screen in this role's navigation courses calls.

Generated from docs/diagrams/n8n/reep-feature-inventory.json by tools/diagrams/render_roles_crud_page.py , with the same rules as the n8n workflow reep-roles-features-crud.n8n.json .